

2. CPU

The RXv3 CPU is based on the RXv3 instruction set architecture. Its instruction processing efficiency has been improved relative to that of the RXv2 CPU, so it delivers higher performance.

The RXv3 instruction set architecture (RXv3) provides upward compatibility from the RXv2 instruction set architecture (RXv2) and the RXv1 instruction set architecture (RXv1).

- Adoption of variable-length instruction format
The CPU has short formats for frequently used instructions, facilitating the development of efficient programs that take up less memory.
- Powerful instruction set
DSP instructions and floating-point operation instructions realize high-speed arithmetic processing.
- Versatile addressing modes
The CPU has versatile addressing modes, with register-register operations, register-memory operations, and bitwise operations included. Data transfer between memory locations is also possible.

2.1 Features

- Minimum instruction execution rate: One clock cycle
- Address space: 4-Gbyte linear addresses
- Register set of the CPU
General purpose: Sixteen 32-bit registers
Control: Ten 32-bit registers
Accumulator: Two 72-bit registers
- Variable-length instruction format (lengths from one to eight bytes)
- 113 instructions
Standard provided instructions: 111
Basic instructions: 77
Single-precision floating point instructions: 11
DSP instructions: 23
Instructions for register bank save function: 2
- Processor modes
Supervisor mode and user mode
- Vector tables
Exception vector table and interrupt vector table
- Memory protection unit
- Data arrangement
Selectable as little endian or big endian
- On-chip double-precision floating point coprocessor
Double-precision floating-point processing instructions: 21

2.2 Register Set of the CPU

The CPU has sixteen general-purpose registers, ten control registers, and two accumulator used for DSP instructions.

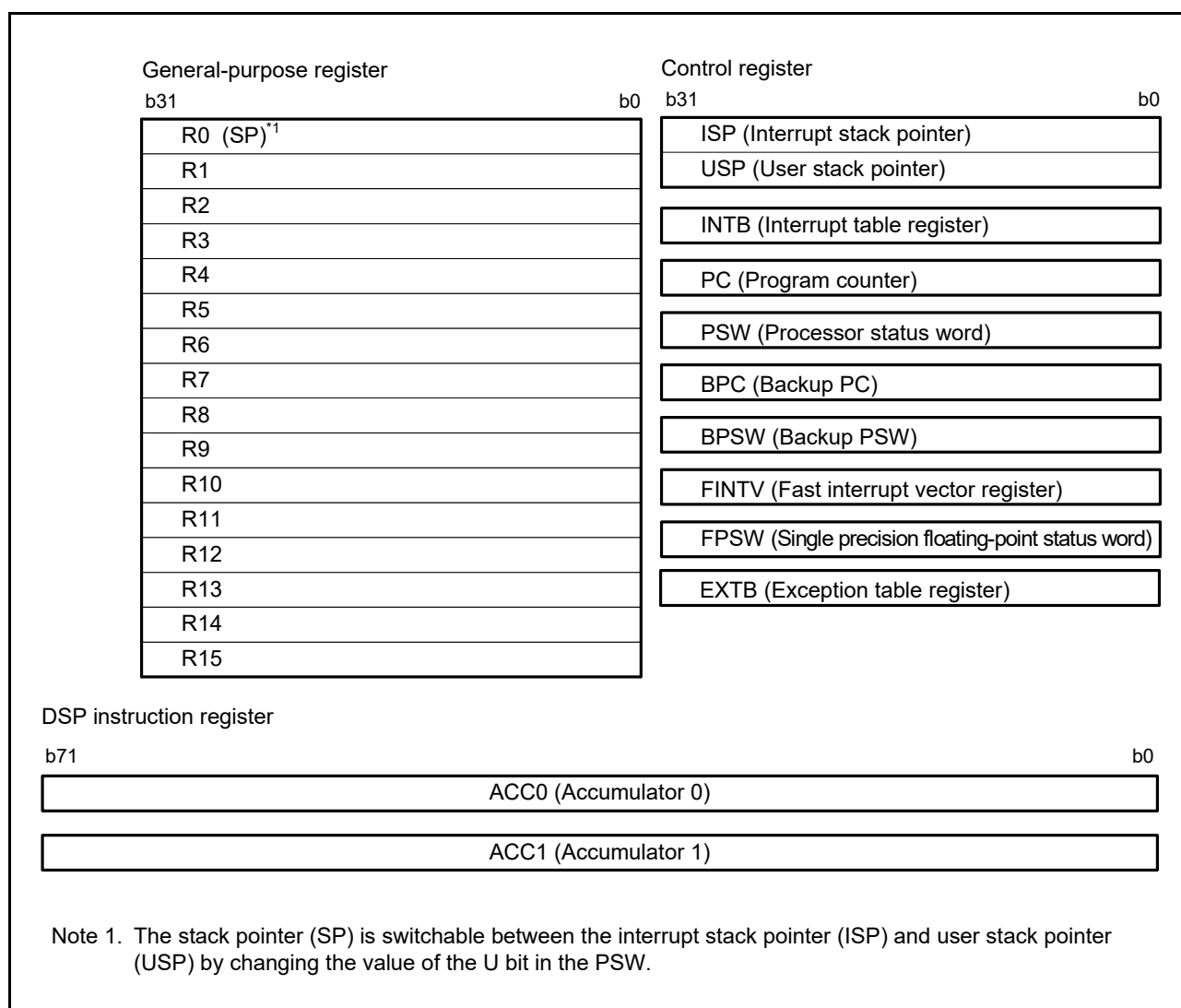


Figure 2.1 Register Set of the CPU

2.2.1 General-Purpose Registers (R0 to R15)

This CPU has sixteen 32-bit general-purpose registers (R0 to R15). R0 to R15 can be used as data registers or address registers.

R0, a general-purpose register, also functions as the stack pointer (SP).

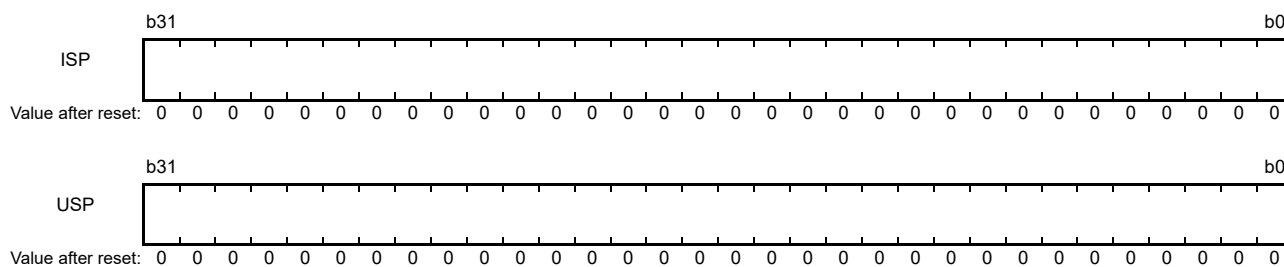
The stack pointer is switched to operate as the interrupt stack pointer (ISP) or user stack pointer (USP) by the value of the stack pointer select bit (U) in the processor status word (PSW).

2.2.2 Control Registers

This CPU has the following ten control registers.

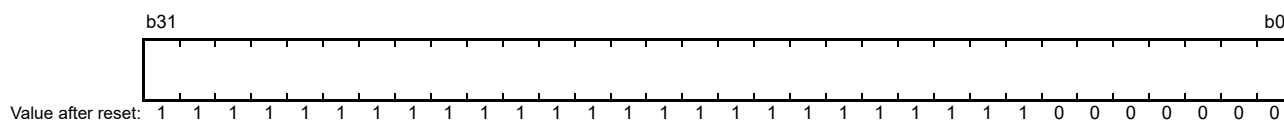
- Interrupt stack pointer (ISP)
- User stack pointer (USP)
- Exception table register (EXTB)
- Interrupt table register (INTB)
- Program counter (PC)
- Processor status word (PSW)
- Backup PC (BPC)
- Backup PSW (BPSW)
- Fast interrupt vector register (FINTV)
- Single-precision floating-point status word (FPSW)

2.2.2.1 Interrupt Stack Pointer (ISP)/User Stack Pointer (USP)



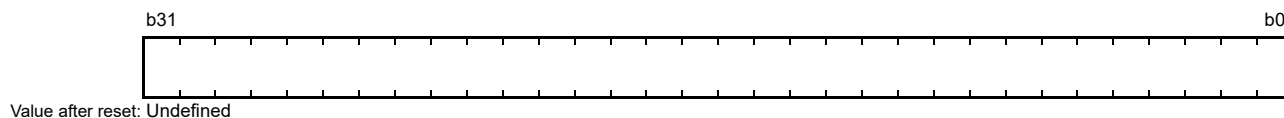
The stack pointer (SP) can be either of two types, the interrupt stack pointer (ISP) or the user stack pointer (USP). Whether the stack pointer operates as the ISP or USP depends on the value of the stack pointer select bit (U) in the processor status word (PSW).

2.2.2.2 Exception Table Register (EXTB)



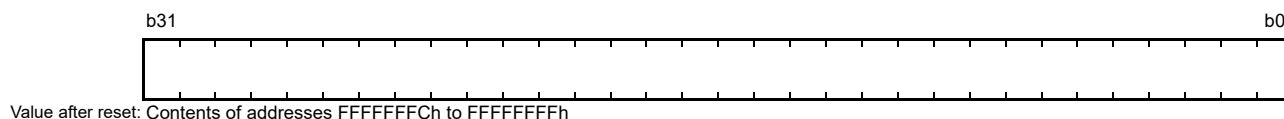
The exception table register (EXTB) specifies the address where the exception vector table starts.

2.2.2.3 Interrupt Table Register (INTB)



The interrupt table register (INTB) specifies the address where the interrupt vector table starts.

2.2.2.4 Program Counter (PC)



The program counter (PC) indicates the address of the instruction being executed.

2.2.2.5 Processor Status Word (PSW)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
—	—	—	—	IPL[3:0]				—	—	—	PM	—	—	U	I
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
—	—	—	—	—	—	—	—	—	—	—	—	O	S	Z	C
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Bit Name	Description	R/W
b0	C	Carry Flag	0: No carry has occurred. 1: A carry has occurred.	R/W
b1	Z	Zero Flag	0: Result is non-zero. 1: Result is 0.	R/W
b2	S	Sign Flag	0: Result is a positive value or 0. 1: Result is a negative value.	R/W
b3	O	Overflow Flag	0: No overflow has occurred. 1: An overflow has occurred.	R/W
b15 to b4	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b16	I*1	Interrupt Enable	0: Interrupt disabled. 1: Interrupt enabled.	R/W
b17	U*1	Stack Pointer Select	0: Interrupt stack pointer (ISP) is selected. 1: User stack pointer (USP) is selected.	R/W
b19, b18	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b20	PM*1, *2, *3	Processor Mode Select	0: Supervisor mode is selected. 1: User mode is selected.	R/W
b23 to b21	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b27 to b24	IPL[3:0]*1	Processor Interrupt Priority Level	b27 b24 0 0 0 0: Priority level 0 (lowest) 0 0 0 1: Priority level 1 0 0 1 0: Priority level 2 0 0 1 1: Priority level 3 0 1 0 0: Priority level 4 0 1 0 1: Priority level 5 0 1 1 0: Priority level 6 0 1 1 1: Priority level 7 1 0 0 0: Priority level 8 1 0 0 1: Priority level 9 1 0 1 0: Priority level 10 1 0 1 1: Priority level 11 1 1 0 0: Priority level 12 1 1 0 1: Priority level 13 1 1 1 0: Priority level 14 1 1 1 1: Priority level 15 (highest)	R/W
b31 to b28	—	Reserved	These bits are read as 0. The write value should be 0.	R/W

Note 1. In user mode, writing to the IPL[3:0], PM, U, and I bits by an MVTC or a POPC instruction is ignored. Writing to the IPL[3:0] bits by an MVTIPL instruction generates a privileged instruction exception.

Note 2. In supervisor mode, writing to the PM bit by an MVTC or a POPC instruction is ignored, but writing to the other bits is possible.

Note 3. Switching from supervisor mode to user mode requires execution of an RTE instruction after having set the PSW.PM bit saved on the stack to 1 or executing an RTFI instruction after having set the BPSW.PM bit to 1.

The processor status word (PSW) indicates the results of instruction execution or the state of the CPU.

C Flag (Carry Flag)

This flag retains the state of the bit after a carry, borrow, or shift-out has occurred.

Z Flag (Zero Flag)

This flag is set to 1 if the result of an operation is 0; otherwise its value is set to 0.

S Flag (Sign Flag)

This flag is set to 1 if the result of an operation is negative; otherwise its value is set to 0.

O Flag (Overflow Flag)

This flag is set to 1 if the result of an operation overflows; otherwise its value is set to 0.

I Bit (Interrupt Enable)

This bit enables interrupt requests. When a WAIT instruction is executed, the value of this bit becomes 1. It becomes 0 when an exception is accepted.

U Bit (Stack Pointer Select)

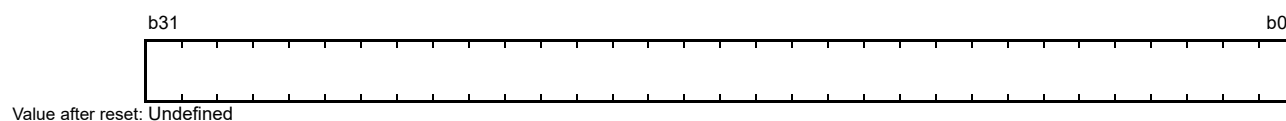
This bit specifies the stack pointer as either the ISP or USP. When an exception request is accepted, this bit is set to 0. When the processor mode is switched from supervisor mode to user mode, this bit is set to 1.

PM Bit (Processor Mode Select)

This bit specifies the processor mode. When an exception is accepted, the value of this bit becomes 0.

IPL[3:0] Bits (Processor Interrupt Priority Level)

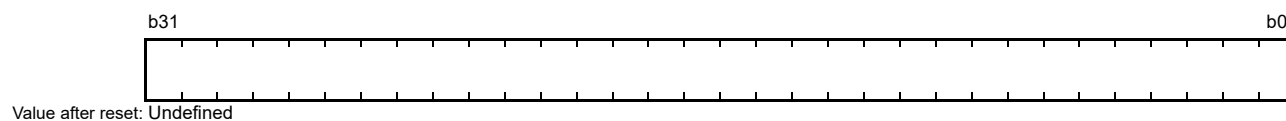
The IPL[3:0] bits specify the processor interrupt priority level as one of sixteen levels from zero to fifteen, wherein priority level zero is the lowest and priority level fifteen the highest. When the priority level of a requested interrupt is higher than the processor interrupt priority level, the interrupt is enabled. Setting the IPL[3:0] bits to level fifteen (Fh) disables all interrupt requests. The IPL[3:0] bits are set to level fifteen (Fh) when a non-maskable interrupt is generated. When interrupts in general are generated, the bits are set to the priority levels of accepted interrupts.

2.2.2.6 Backup PC (BPC)

The backup PC (BPC) is provided to speed up response to interrupts.

After a fast interrupt has been generated, the contents of the program counter (PC) are saved in the BPC register.

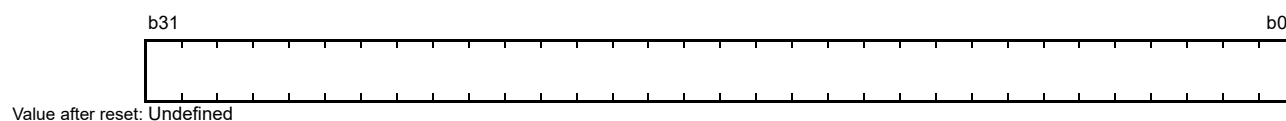
2.2.2.7 Backup PSW (BPSW)



The backup PSW (BPSW) is provided to speed up response to interrupts.

After a fast interrupt has been generated, the contents of the processor status word (PSW) are saved in the BPSW. The allocation of bits in the BPSW corresponds to that in the PSW.

2.2.2.8 Fast Interrupt Vector Register (FINTV)



The fast interrupt vector register (FINTV) is provided to speed up response to interrupts.

The FINTV register specifies a branch destination address when a fast interrupt has been generated.

2.2.2.9 Single-Precision Floating-Point Status Word (FPSW)

b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
FS	FX	FU	FZ	FO	FV	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
—	EX	EU	EZ	EO	EV	—	DN	CE	CX	CU	CZ	CO	CV	RM[1:0]	—
Value after reset:	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Symbol	Bit Name	Description	R/W
b1, b0	RM[1:0]	Single-Precision Floating-Point Rounding-Mode Setting	b1 b0 0 0: Rounding towards the nearest value 0 1: Rounding towards 0 1 0: Rounding towards $+\infty$ 1 1: Rounding towards $-\infty$	R/W
b2	CV	Invalid Operation Cause Flag	0: No invalid operation has been encountered. 1: Invalid operation has been encountered.	R/(W) *1
b3	CO	Overflow Cause Flag	0: No overflow has occurred. 1: Overflow has occurred.	R/(W) *1
b4	CZ	Division-by-Zero Cause Flag	0: No division-by-zero has occurred. 1: Division-by-zero has occurred.	R/(W) *1
b5	CU	Underflow Cause Flag	0: No underflow has occurred. 1: Underflow has occurred.	R/(W) *1
b6	CX	Inexact Cause Flag	0: No inexact exception has been generated. 1: Inexact exception has been generated.	R/(W) *1
b7	CE	Unimplemented Processing Cause Flag	0: No unimplemented processing has been encountered. 1: Unimplemented process has been encountered.	R/(W) *1
b8	DN	0 Flush Bit of Denormalized Number	0: A denormalized number is handled as a denormalized number. 1: A denormalized number is handled as 0.*2	R/W
b9	—	Reserved	This bit is read as 0. The write value should be 0.	R/W
b10	EV	Invalid Operation Exception Enable	0: Invalid operation exception is masked. 1: Invalid operation exception is enabled.	R/W
b11	EO	Overflow Exception Enable	0: Overflow exception is masked. 1: Overflow exception is enabled.	R/W
b12	EZ	Division-by-Zero Exception Enable	0: Division-by-zero exception is masked. 1: Division-by-zero exception is enabled.	R/W
b13	EU	Underflow Exception Enable	0: Underflow exception is masked. 1: Underflow exception is enabled.	R/W
b14	EX	Inexact Exception Enable	0: Inexact exception is masked. 1: Inexact exception is enabled.	R/W
b25 to b15	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b26	FV*3	Invalid Operation Flag	0: No invalid operation has been encountered. 1: Invalid operation has been encountered.*8	R/W
b27	FO*4	Overflow Flag	0: No overflow has occurred. 1: Overflow has occurred.*8	R/W
b28	FZ*5	Division-by-Zero Flag	0: No division-by-zero has occurred. 1: Division-by-zero has occurred.*8	R/W
b29	FU*6	Underflow Flag	0: No underflow has occurred. 1: Underflow has occurred.*8	R/W
b30	FX*7	Inexact Flag	0: No inexact exception has been generated. 1: Inexact exception has been generated.*8	R/W
b31	FS	Single-Precision Floating-Point Error Summary Flag	This bit reflects the logical OR of the FU, FZ, FO, and FV flags.	R

- Note 1. Writing 0 to the bit clears it. Writing 1 to the bit does not affect its value.
- Note 2. Positive denormalized numbers are treated as +0, negative denormalized numbers as -0.
- Note 3. When the EV bit is set to 0, the FV flag is enabled.
- Note 4. When the EO bit is set to 0, the FO flag is enabled.
- Note 5. When the EZ bit is set to 0, the FZ flag is enabled.
- Note 6. When the EU bit is set to 0, the FU flag is enabled.
- Note 7. When the EX bit is set to 0, the FX flag is enabled.
- Note 8. Once the bit has been set to 1, this value is retained until it is set to 0 by software.

The single-precision floating-point status word (FPSW) indicates the results of single-precision floating-point arithmetic operations.

When the corresponding exception handling enable bits (E_j) are set to enable processing of the exceptions ($E_j = 1$), the C_j flags can be used by the exception handling routine to identify the source of that exception. If handling of an exception is masked ($E_j = 0$), the F_j flag can be used to check for the generation of the exception at the end of a sequence of processing. The F_j flags operate in an accumulative fashion ($j = X, U, Z, O, \text{ or } V$).

RM[1:0] Bits (Single-Precision Floating-Point Rounding-Mode Setting)

These bits specify the single-precision floating-point rounding-mode.

Explanation of Single-Precision Floating-Point Rounding Modes

- Rounding towards the nearest value (the default behavior): An inexact result is rounded to the available value that is closest to the result of a hypothetical calculation with infinite precision. If two available values are equally close, rounding is to the even alternative.
- Rounding towards 0: An inexact result is rounded to the smallest available absolute value, i.e. in the direction of zero (simple truncation).
- Rounding towards $+\infty$: An inexact result is rounded to the nearest available value in the direction of positive infinity.
- Rounding towards $-\infty$: An inexact result is rounded to the nearest available value in the direction of negative infinity.

- (1) Rounding to the nearest value is specified as the default mode and returns the most accurate value.
- (2) Modes such as rounding towards 0, rounding towards $+\infty$, and rounding towards $-\infty$ are used to ensure precision when interval arithmetic is employed.

CV Flag (Invalid Operation Cause Flag), CO Flag (Overflow Cause Flag), CZ Flag (Division-by-Zero Cause Flag), CU Flag (Underflow Cause Flag), CX Flag (Inexact Cause Flag), and CE Flag (Unimplemented Processing Cause Flag)

Single-precision floating-point exceptions include the five specified in the IEEE754 standard, namely overflow, underflow, inexact, division-by-zero, and invalid operation. For a further single-precision floating-point exception that is generated upon detection of unimplemented processing, the corresponding flag (CE) is set to 1.

- If an exception or processing that is not implemented is not encountered in the execution of a single-precision floating-point arithmetic instruction, the corresponding flags become 0.
- When 0 is written to the bit by the MVTC and POPC instructions, the bit is set to 0; the bit retains the previous value when 1 is written by the instruction.

DN Bit (0 Flush Bit of Denormalized Number)

When this bit is set to 0, a denormalized number is handled as a denormalized number. When this bit is set to 1, a denormalized number is handled as 0.

EV Bit (Invalid Operation Exception Enable), EO Bit (Overflow Exception Enable), EZ Bit (Division-by-Zero Exception Enable), EU Bit (Underflow Exception Enable), and EX Bit (Inexact Exception Enable)

When any of five single-precision floating-point exceptions specified in the IEEE754 standard is generated by the single-precision floating-point operation instruction, the bit decides whether the CPU will start handling the exception. When the bit is set to 0, the exception handling is masked; when the bit is set to 1, the exception handling is enabled.

FV Flag (Invalid Operation Flag), FO Flag (Overflow Flag), FZ Flag (Division-by-Zero Flag), FU Flag (Underflow Flag), and FX Flag (Inexact Flag)

While the exception handling enable bit (Ej) is 0 (exception handling is masked), if any of five single-precision floating-point exceptions specified in the IEEE754 standard is generated, the corresponding bit is set to 1.

- When Ej is 1 (exception handling is enabled), the value of the flag remains.
- When the corresponding flag is set to 1, it remains 1 until it is set to 0 by software (accumulation flag).

FS Flag (Single-Precision Floating-Point Error Summary Flag)

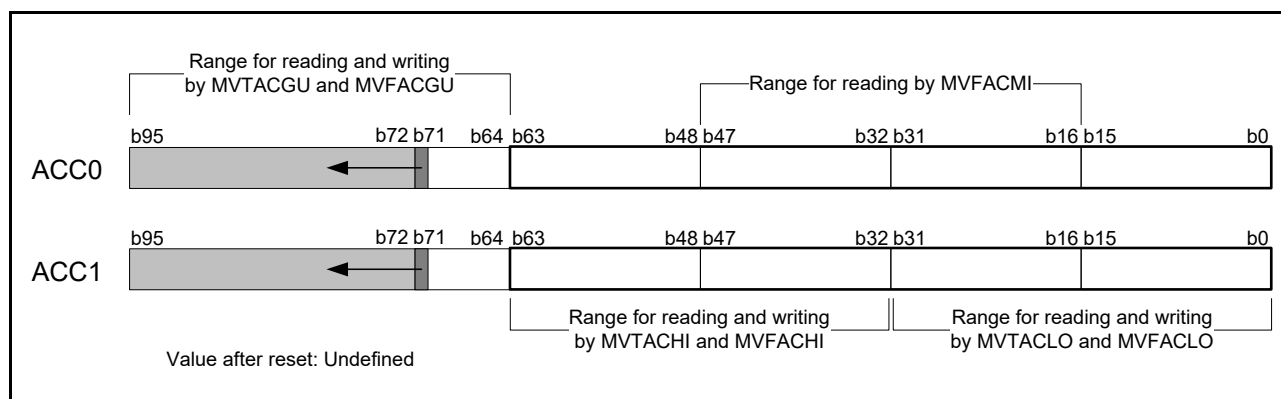
This bit reflects the logical OR of the FU, FZ, FO, and FV flags.

2.2.3 Accumulator

The accumulator (ACC0 or ACC1) is a 72-bit register used for DSP instructions. The accumulator is handled as a 96-bit register for reading and writing. At this time, when bits 95 to 72 of the accumulator are read, the value where the value of bit 71 is sign extended is read. Writing to bits 95 to 72 of the accumulator is ignored. ACC0 is also used for the multiply and multiply-and-accumulate instructions; EMUL, EMULU, FMUL, MUL, and RMPA, in which case the prior value in ACC0 is modified by execution of the instruction.

Use the MVTACGU, MVTACHI, and MVTACLO instructions for writing to the accumulator. The MVTACGU, MVTACHI, and MVTACLO instructions write data to bits 95 to 64, the higher-order 32 bits (bits 63 to 32), and the lower-order 32 bits (bits 31 to 0), respectively.

Use the MVFACGU, MVFACHI, MVFACMI, and MVFACLO instructions for reading data from the accumulator. The MVFACGU, MVFACHI, MVFACMI, and MVFACLO instructions read data from the guard bits (bits 95 to 64), higher-order 32 bits (bits 63 to 32), the middle 32 bits (bits 31 to 0), and the lower-order 32 bits (bits 31 to 0), respectively.



Note: The value of bit 71 is sign extended for bits 95 to 72 and the extended value is always read. Writing to this area is ignored.

2.3 Processor Mode

The CPU supports two processor modes, supervisor and user. These processor modes and the memory protection function enable the realization of a hierarchical CPU resource protection and memory protection mechanism. Each processor mode imposes a level on rights of access to memory and the instructions that can be executed. Supervisor mode carries greater rights than user mode. The initial state after a reset is supervisor mode.

2.3.1 Supervisor Mode

In supervisor mode, all CPU resources are accessible and all instructions are available. However, writing to the processor mode select bit (PM) in the processor status word (PSW) by executing an MVTC or a POPC instruction will be ignored. For details on how to write to the PM bit, refer to section 2.2.2.5, Processor Status Word (PSW).

2.3.2 User Mode

In user mode, write access to the CPU resources listed below is restricted. The restriction applies to any instruction capable of write access.

- Some bits (bits IPL[3:0], PM, U, and I) in the processor status word (PSW)
- Interrupt stack pointer (ISP)
- Exception table register (EXTB)
- Interrupt table register (INTB)
- Backup PSW (BPSW)
- Backup PC (BPC)
- Fast interrupt vector register (FINTV)

2.3.3 Privileged Instruction

Privileged instructions can only be executed in supervisor mode. Executing a privileged instruction in user mode produces a privileged instruction exception. Privileged instructions include the RTFI, MVTIPL, RTE, WAIT, SAVE, and RSTR instructions.

2.3.4 Switching Between Processor Modes

Manipulating the processor mode select bit (PM) in the processor status word (PSW) switches the processor mode. However, rewriting to the PM bit by executing an MVTC or a POPC instruction is prohibited. Switch the processor mode by following the procedures described below.

(1) Switching from user mode to supervisor mode

After an exception has been generated, the PSW.PM bit is set to 0 and the CPU switches to supervisor mode. The hardware pre-processing is executed in supervisor mode. The state of the processor mode before the exception was generated is retained in the copy of PSW.PM bit is saved on the stack.

(2) Switching from supervisor mode to user mode

Executing an RTE instruction when the value of the copy of the PSW.PM bit that has been preserved on the stack is 1 or an RTFI instruction when the value of the copy of the PSW.PM bit that has been preserved in the backup PSW (BPSW) is 1 causes a transition to user mode. In the transition to user mode, the value of the stack pointer designation bit (the U bit in the PSW) becomes 1.

2.4 Data Types

The CPU can handle four types of data: integer, single-precision floating-point number, bit, and string.
For details, refer to RX Family RXv3 Instruction Set Architecture User’s Manual: Software.

2.4.1 Integer

An integer can be signed or unsigned. For signed integers, negative values are represented by two’s complements.

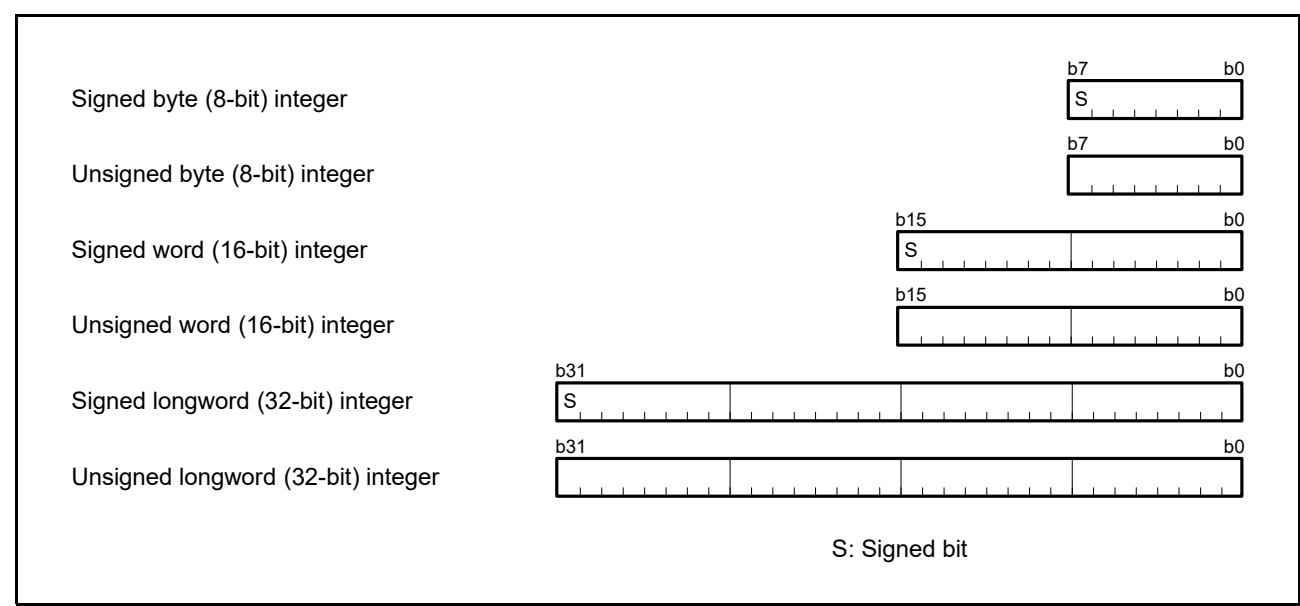


Figure 2.2 Integer

2.4.2 Single-Precision Floating-Point Numbers

The single-precision floating-point number is compliant with that specified in the IEEE754 standard; operands of this type can be used in eleven single-precision floating-point operation instructions: FADD, FCMP, FDIV, FMUL, FSQRT, FSUB, FTOI, FTOU, ITOF, ROUND, and UTOF.

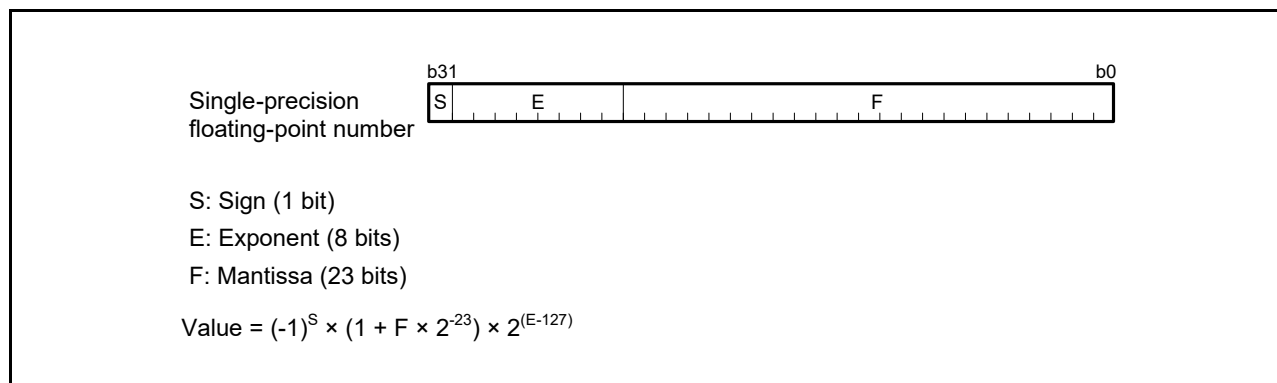


Figure 2.3 Single-Precision Floating-Point Number

The single-precision floating-point number can represent the values listed below.

- 0 < E < 255 (normal numbers)
- E = 0 and F = 0 (signed zero)
- E = 0 and F > 0 (denormalized numbers)*1
- E = 255 and F = 0 (infinity)
- E = 255 and F > 0 (NaN: Not-a-Number)

Note 1. The number is treated as 0 when the FPSW.DN bit is 1. When the DN bit is 0, an unimplemented processing exception is generated.

2.4.3 Bitwise Operations

Five bit-manipulation instructions are provided for bitwise operations: BCLR, BMCnd, BNOT, BSET, and BTST.

A bit in a register is specified as the destination register and a bit number in the range from 31 to 0.

A bit in memory is specified as the destination address and a bit number from 7 to 0. The addressing modes available to specify addresses are register indirect and register relative.

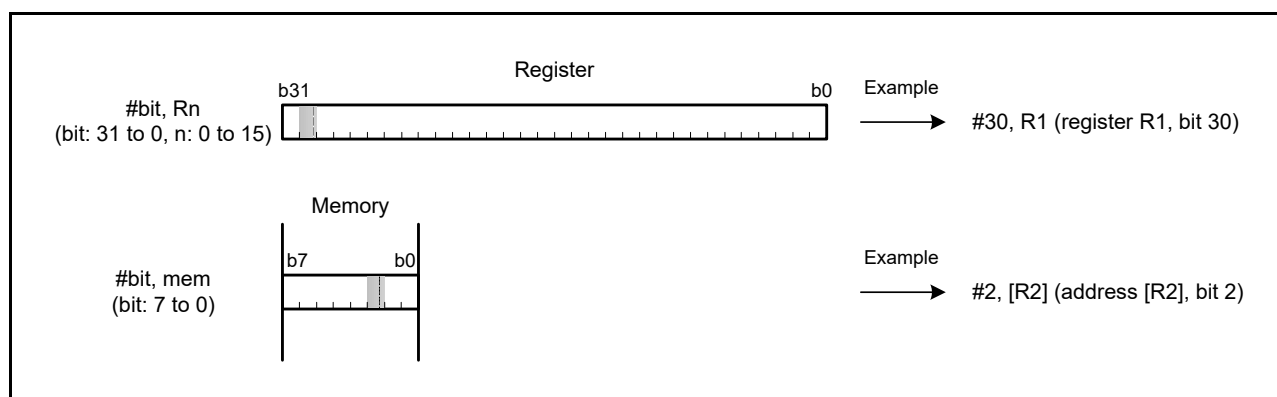


Figure 2.4 Bit

2.4.4 Strings

The string data type consists of an arbitrary number of consecutive byte (8-bit), word (16-bit), or longword (32-bit) units. Seven string manipulation instructions are provided for use with strings: SCMPU, SMOVB, SMOVF, SMOVU, SSTR, SUNTIL, and SWHILE.

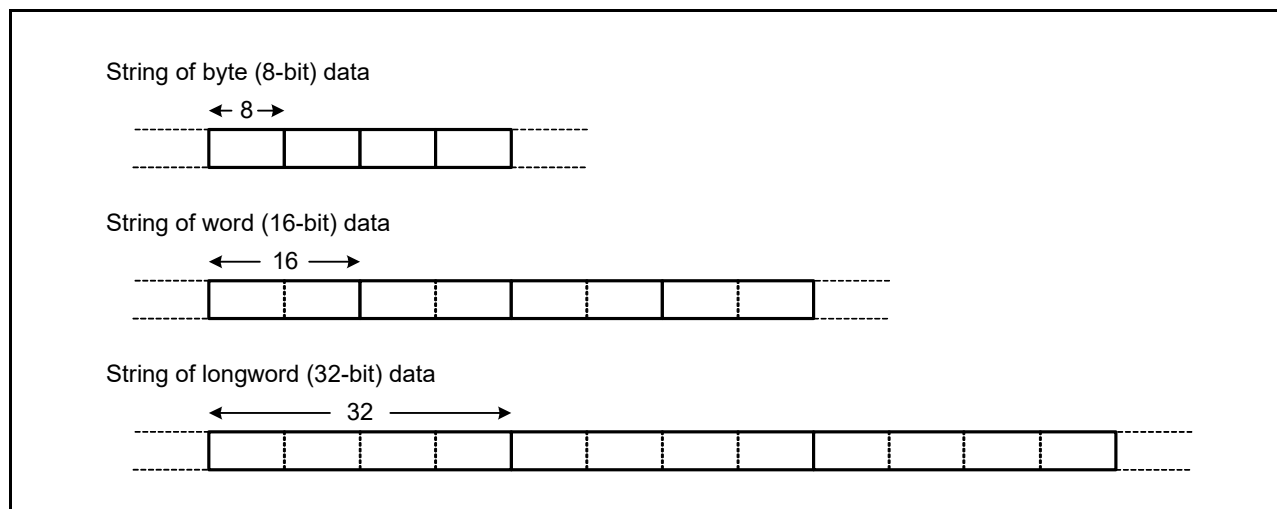


Figure 2.5 String

2.5 Endian

For the CPU, instructions are little endian, but the treatment of data is selectable as little or big endian.

2.5.1 Switching the Endian

As arrangements of bytes, this MCU supports both big endian, where the higher-order byte (MSB) is at location 0, and little endian, where the lower-order byte (LSB) is at location 0.

For details on the endian setting, see section 3, Operating Modes.

Operations for access differ according to the endian setting and, depending on the instruction, whether 8-, 16- or 32-bit access has been selected. Operations for access in the various possible cases are described in Table 2.1 to Table 2.12.

In the tables,

- LL indicates bits D7 to D0 of the general-purpose register,
- LH indicates bits D15 to D8 of the general-purpose register,
- HL indicates bits D23 to D16 of the general-purpose register, and
- HH indicates bits D31 to D24 of the general-purpose register.

	D31 to D24	D23 to D16	D15 to D8	D7 to D0
General purpose register: Rm	HH	HL	LH	LL

Table 2.1 32-Bit Read Operations when Little Endian has been Selected

Operation Address of src	Reading a 32-bit unit from address 0	Reading a 32-bit unit from address 1	Reading a 32-bit unit from address 2	Reading a 32-bit unit from address 3	Reading a 32-bit unit from address 4
Address 0	Transfer to LL	—	—	—	—
Address 1	Transfer to LH	Transfer to LL	—	—	—
Address 2	Transfer to HL	Transfer to LH	Transfer to LL	—	—
Address 3	Transfer to HH	Transfer to HL	Transfer to LH	Transfer to LL	—
Address 4	—	Transfer to HH	Transfer to HL	Transfer to LH	Transfer to LL
Address 5	—	—	Transfer to HH	Transfer to HL	Transfer to LH
Address 6	—	—	—	Transfer to HH	Transfer to HL
Address 7	—	—	—	—	Transfer to HH

Table 2.2 32-Bit Read Operations when Big Endian has been Selected

Operation Address of src	Reading a 32-bit unit from address 0	Reading a 32-bit unit from address 1	Reading a 32-bit unit from address 2	Reading a 32-bit unit from address 3	Reading a 32-bit unit from address 4
Address 0	Transfer to HH	—	—	—	—
Address 1	Transfer to HL	Transfer to HH	—	—	—
Address 2	Transfer to LH	Transfer to HL	Transfer to HH	—	—
Address 3	Transfer to LL	Transfer to LH	Transfer to HL	Transfer to HH	—
Address 4	—	Transfer to LL	Transfer to LH	Transfer to HL	Transfer to HH
Address 5	—	—	Transfer to LL	Transfer to LH	Transfer to HL
Address 6	—	—	—	Transfer to LL	Transfer to LH
Address 7	—	—	—	—	Transfer to LL

Table 2.3 32-Bit Write Operations when Little Endian has been Selected

Operation Address of dest	Writing a 32-bit unit to address 0	Writing a 32-bit unit to address 1	Writing a 32-bit unit to address 2	Writing a 32-bit unit to address 3	Writing a 32-bit unit to address 4
Address 0	Transfer from LL	—	—	—	—
Address 1	Transfer from LH	Transfer from LL	—	—	—
Address 2	Transfer from HL	Transfer from LH	Transfer from LL	—	—
Address 3	Transfer from HH	Transfer from HL	Transfer from LH	Transfer from LL	—
Address 4	—	Transfer from HH	Transfer from HL	Transfer from LH	Transfer from LL
Address 5	—	—	Transfer from HH	Transfer from HL	Transfer from LH
Address 6	—	—	—	Transfer from HH	Transfer from HL
Address 7	—	—	—	—	Transfer from HH

Table 2.4 32-Bit Write Operations when Big Endian has been Selected

Operation Address of dest	Writing a 32-bit unit to address 0	Writing a 32-bit unit to address 1	Writing a 32-bit unit to address 2	Writing a 32-bit unit to address 3	Writing a 32-bit unit to address 4
Address 0	Transfer from HH	—	—	—	—
Address 1	Transfer from HL	Transfer from HH	—	—	—
Address 2	Transfer from LH	Transfer from HL	Transfer from HH	—	—
Address 3	Transfer from LL	Transfer from LH	Transfer from HL	Transfer from HH	—
Address 4	—	Transfer from LL	Transfer from LH	Transfer from HL	Transfer from HH
Address 5	—	—	Transfer from LL	Transfer from LH	Transfer from HL
Address 6	—	—	—	Transfer from LL	Transfer from LH
Address 7	—	—	—	—	Transfer from LL

Table 2.5 16-Bit Read Operations when Little Endian has been Selected

Operation Address of src	Reading a 16-bit unit from address 0	Reading a 16-bit unit from address 1	Reading a 16-bit unit from address 2	Reading a 16-bit unit from address 3	Reading a 16-bit unit from address 4	Reading a 16-bit unit from address 5	Reading a 16-bit unit from address 6
Address 0	Transfer to LL	—	—	—	—	—	—
Address 1	Transfer to LH	Transfer to LL	—	—	—	—	—
Address 2	—	Transfer to LH	Transfer to LL	—	—	—	—
Address 3	—	—	Transfer to LH	Transfer to LL	—	—	—
Address 4	—	—	—	Transfer to LH	Transfer to LL	—	—
Address 5	—	—	—	—	Transfer to LH	Transfer to LL	—
Address 6	—	—	—	—	—	Transfer to LH	Transfer to LL
Address 7	—	—	—	—	—	—	Transfer to LH

Table 2.6 16-Bit Read Operations when Big Endian has been Selected

Operation Address of src	Reading a 16-bit unit from address 0	Reading a 16-bit unit from address 1	Reading a 16-bit unit from address 2	Reading a 16-bit unit from address 3	Reading a 16-bit unit from address 4	Reading a 16-bit unit from address 5	Reading a 16-bit unit from address 6
Address 0	Transfer to LH	—	—	—	—	—	—
Address 1	Transfer to LL	Transfer to LH	—	—	—	—	—
Address 2	—	Transfer to LL	Transfer to LH	—	—	—	—
Address 3	—	—	Transfer to LL	Transfer to LH	—	—	—
Address 4	—	—	—	Transfer to LL	Transfer to LH	—	—
Address 5	—	—	—	—	Transfer to LL	Transfer to LH	—
Address 6	—	—	—	—	—	Transfer to LL	Transfer to LH
Address 7	—	—	—	—	—	—	Transfer to LL

Table 2.7 16-Bit Write Operations when Little Endian has been Selected

Operation Address of dest	Writing a 16-bit unit to address 0	Writing a 16-bit unit to address 1	Writing a 16-bit unit to address 2	Writing a 16-bit unit to address 3	Writing a 16-bit unit to address 4	Writing a 16-bit unit to address 5	Writing a 16-bit unit to address 6
Address 0	Transfer from LL	—	—	—	—	—	—
Address 1	Transfer from LH	Transfer from LL	—	—	—	—	—
Address 2	—	Transfer from LH	Transfer from LL	—	—	—	—
Address 3	—	—	Transfer from LH	Transfer from LL	—	—	—
Address 4	—	—	—	Transfer from LH	Transfer from LL	—	—
Address 5	—	—	—	—	Transfer from LH	Transfer from LL	—
Address 6	—	—	—	—	—	Transfer from LH	Transfer from LL
Address 7	—	—	—	—	—	—	Transfer from LH

Table 2.8 16-Bit Write Operations when Big Endian has been Selected

Operation Address of dest	Writing a 16-bit unit to address 0	Writing a 16-bit unit to address 1	Writing a 16-bit unit to address 2	Writing a 16-bit unit to address 3	Writing a 16-bit unit to address 4	Writing a 16-bit unit to address 5	Writing a 16-bit unit to address 6
Address 0	Transfer from LH	—	—	—	—	—	—
Address 1	Transfer from LL	Transfer from LH	—	—	—	—	—
Address 2	—	Transfer from LL	Transfer from LH	—	—	—	—
Address 3	—	—	Transfer from LL	Transfer from LH	—	—	—
Address 4	—	—	—	Transfer from LL	Transfer from LH	—	—
Address 5	—	—	—	—	Transfer from LL	Transfer from LH	—
Address 6	—	—	—	—	—	Transfer from LL	Transfer from LH
Address 7	—	—	—	—	—	—	Transfer from LL

Table 2.9 8-Bit Read Operations when Little Endian has been Selected

Operation Address of src	Reading an 8-bit unit from address 0	Reading an 8-bit unit from address 1	Reading an 8-bit unit from address 2	Reading an 8-bit unit from address 3
Address 0	Transfer to LL	—	—	—
Address 1	—	Transfer to LL	—	—
Address 2	—	—	Transfer to LL	—
Address 3	—	—	—	Transfer to LL

Table 2.10 8-Bit Read Operations when Big Endian has been Selected

Operation Address of src	Reading an 8-bit unit from address 0	Reading an 8-bit unit from address 1	Reading an 8-bit unit from address 2	Reading an 8-bit unit from address 3
Address 0	Transfer to LL	—	—	—
Address 1	—	Transfer to LL	—	—
Address 2	—	—	Transfer to LL	—
Address 3	—	—	—	Transfer to LL

Table 2.11 8-Bit Write Operations when Little Endian has been Selected

Operation Address of dest	Writing an 8-bit unit to address 0	Writing an 8-bit unit to address 1	Writing an 8-bit unit to address 2	Writing an 8-bit unit to address 3
Address 0	Transfer from LL	—	—	—
Address 1	—	Transfer from LL	—	—
Address 2	—	—	Transfer from LL	—
Address 3	—	—	—	Transfer from LL

Table 2.12 8-Bit Write Operations when Big Endian has been Selected

Operation Address of dest	Writing an 8-bit unit to address 0	Writing an 8-bit unit to address 1	Writing an 8-bit unit to address 2	Writing an 8-bit unit to address 3
Address 0	Transfer from LL	—	—	—
Address 1	—	Transfer from LL	—	—
Address 2	—	—	Transfer from LL	—
Address 3	—	—	—	Transfer from LL

2.5.2 Access to I/O Registers

The addresses of I/O registers are fixed, and this is regardless of whether the setting is for little endian or big endian. Accordingly, changes to the endian do not affect access to I/O registers. For the arrangements of I/O registers, refer to the descriptions of registers in the relevant sections.

2.5.3 Notes on Access to I/O Registers

Ensure that access to I/O registers is in accord with the following rules.

- With I/O registers for which a bus width of eight bits is indicated, use instructions having operands of the same width (eight bits). That is, access these registers by using instructions with .B as the size specifier (.size), or with .B or .UB as the size-extension specifier (.memex).
- With I/O registers for which a bus width of 16 bits is indicated, use instructions having operands of the same width (16 bits). That is, access these registers by using instructions with .W as the size specifier (.size), or with .W or .UW as the size-extension specifier (.memex).
- With I/O registers for which a bus width of 32 bits is indicated, use instructions having operands of the same width (32 bits). That is, access these registers by using instructions with .L as the size specifier (.size), or with .L size-extension specifier (.memex).

2.5.4 Data Arrangement

2.5.4.1 Data Arrangement in Registers

Figure 2.6 shows the relation between the sizes of registers and bit numbers.

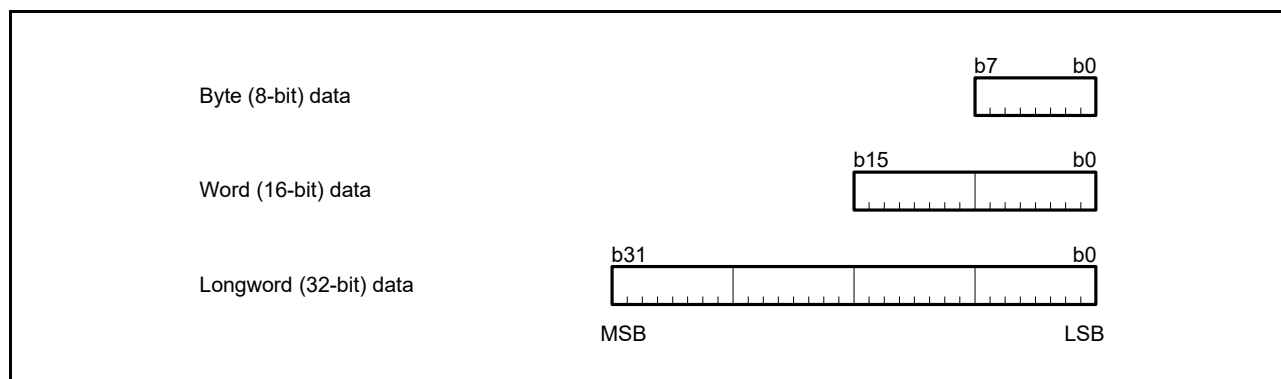


Figure 2.6 Data Arrangement in Registers

2.5.4.2 Data Arrangement in Memory

Data in memory have three sizes: byte (8-bit), word (16-bit), and longword (32-bit). The data arrangement is selectable as little endian or big endian. Figure 2.7 shows the arrangement of data in memory.

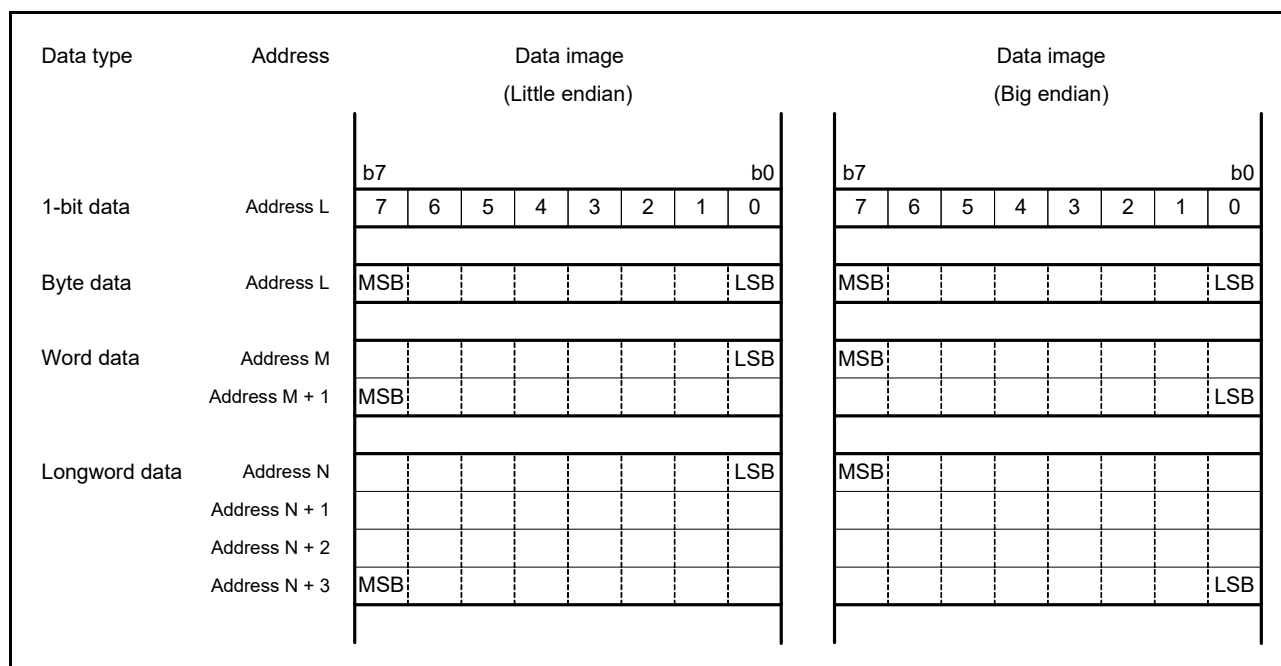


Figure 2.7 Data Arrangement in Memory

2.5.5 Notes on the Allocation of Instruction Codes

The allocation of instruction codes to an external space where the endian differs from that of the chip is prohibited. If the instruction codes are allocated to the external space, they must be allocated to areas where the endian setting is the same as that for the chip.

2.6 Vector Table

There are two types of vector table: exception and interrupt. Each vector in the vector table consists of four bytes and specifies the address where the corresponding exception handling routine starts.

2.6.1 Exception Vector Table

In the exception vector table, the individual vectors for the privileged instruction exception, access exception, address exception, undefined instruction exception, single-precision floating-point exception, and non-maskable interrupt are allocated to the 124-byte area where the value indicated by the exception table register (EXTB) is used as the starting address (ExtBase). The reset vector is always allocated to FFFFFFFCh, regardless of the value of the exception vector table.

Figure 2.8 shows the exception vector table.

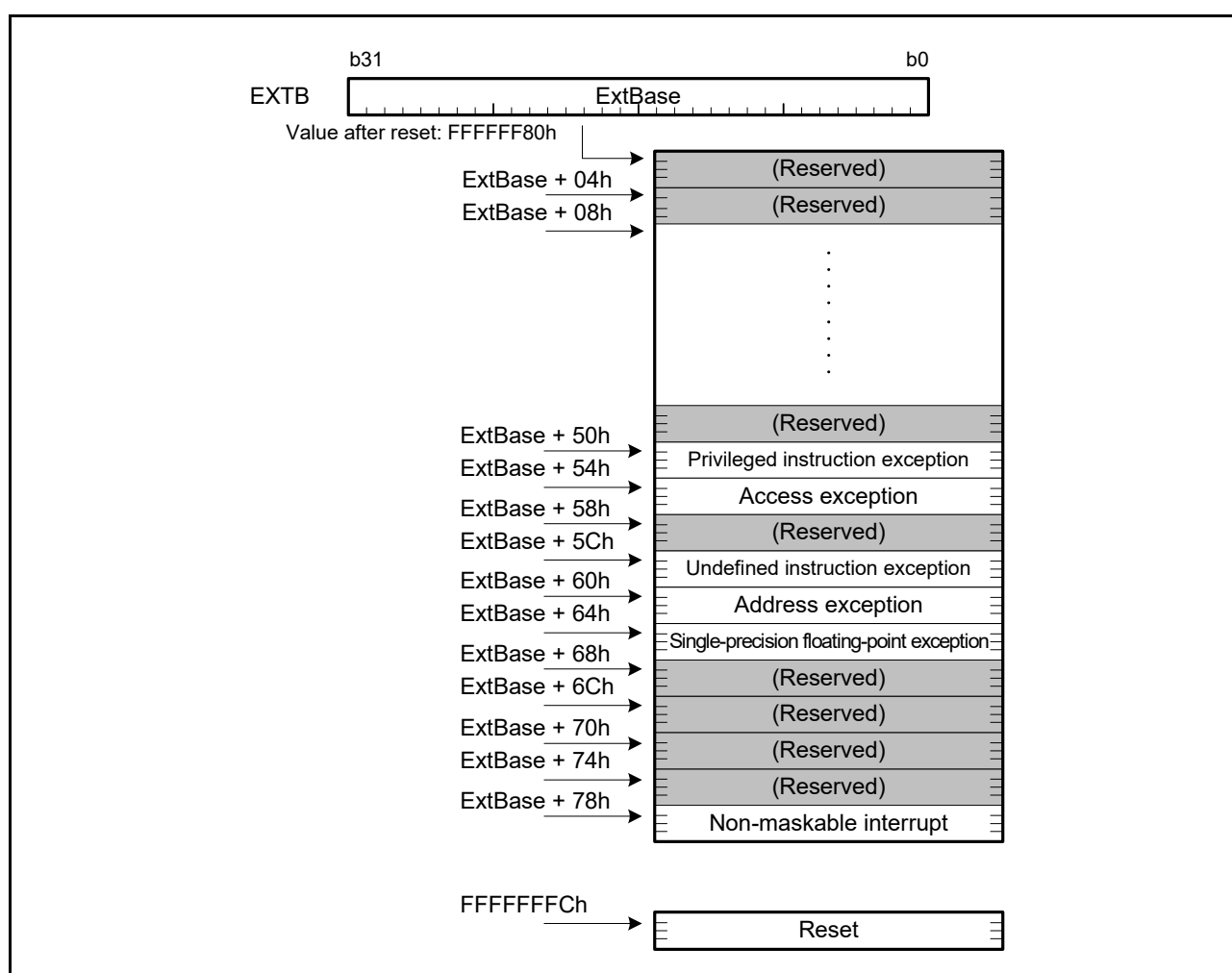


Figure 2.8 Exception Vector Table

2.6.2 Interrupt Vector Table

The address where the interrupt vector table is placed can be adjusted. The table is a 1,024-byte region that contains all vectors for unconditional traps and interrupts and starts at the address (IntBase) specified in the interrupt table register (INTB). Figure 2.9 shows the interrupt vector table.

Each vector in the interrupt vector table has a vector number from 0 to 255. Each of the INT instructions, which act as the sources of unconditional traps, is allocated to the vector that has the same number as is specified as the operand of the instruction itself (from 0 to 255). The BRK instruction is allocated to the vector with number 0. Furthermore, vector numbers (from 0 to 255) are allocated to interrupt requests in a fixed way for each product. For more on interrupt vector numbers, see section 15.3.1, Interrupt Vector Table.

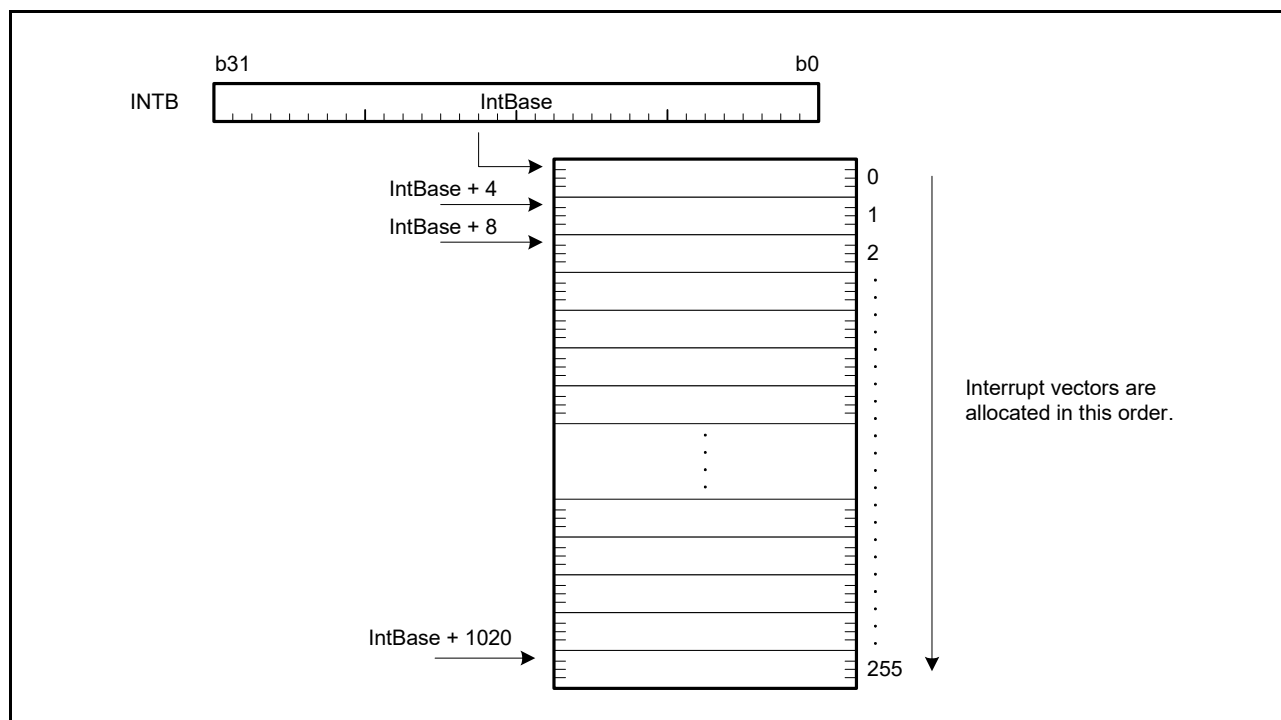


Figure 2.9 Interrupt Vector Table

2.7 Register Bank Save Function

The CPU has dedicated save register banks and functionality for using them for the fast saving and restoring of the values of CPU registers (see Figure 2.10). The save register banks enable the fast collective saving at the start of the exception handling routine and fast collective restoring of register values at the end of the exception handling routine.

The save register banks are only accessible by the SAVE and RSTR instructions, and are independent of the 4-Gbyte address space. Each of the multiple banks is used to save and restore the values of the following CPU registers: all general purpose registers except R0, the USP, FPSW, and accumulators (ACC0, ACC1). Values in the save register banks are undefined after a reset.

A unique number (bank number) is allocated to each save register bank.

The MCU has 16 save register banks, to which the bank numbers 0 to 15 are assigned.

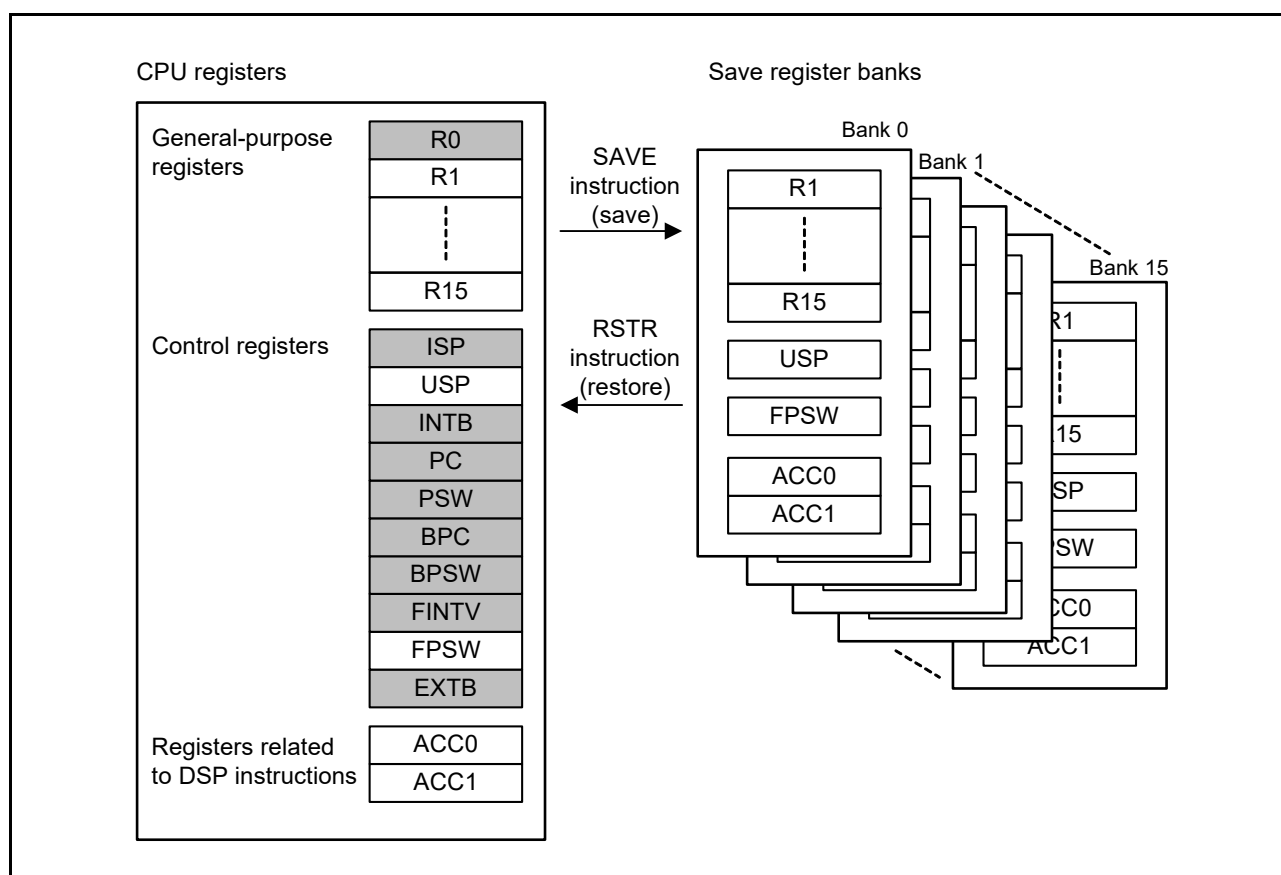


Figure 2.10 Save Register Banks

2.8 Double-Precision Floating-Point Coprocessor

The double-precision floating-point coprocessor operates as a coprocessor of the CPU, and executes double-precision floating-point processing instructions. Using the double-precision floating-point coprocessor considerably accelerates the processing of double-precision floating point arithmetic.

2.8.1 Features

- Double-precision floating-point register set
 - Double-precision floating-point data registers: Sixteen 64-bit registers
 - Double-precision floating-point control registers: Four 32-bit registers
- Double-precision floating-point processing instructions: 21
- Notifying the interrupt controller of double-precision floating-point exceptions

2.8.2 Double-Precision Floating-Point Register Set

The double-precision floating-point coprocessor consists of 16 double-precision floating-point data registers and 4 double-precision floating-point control registers.

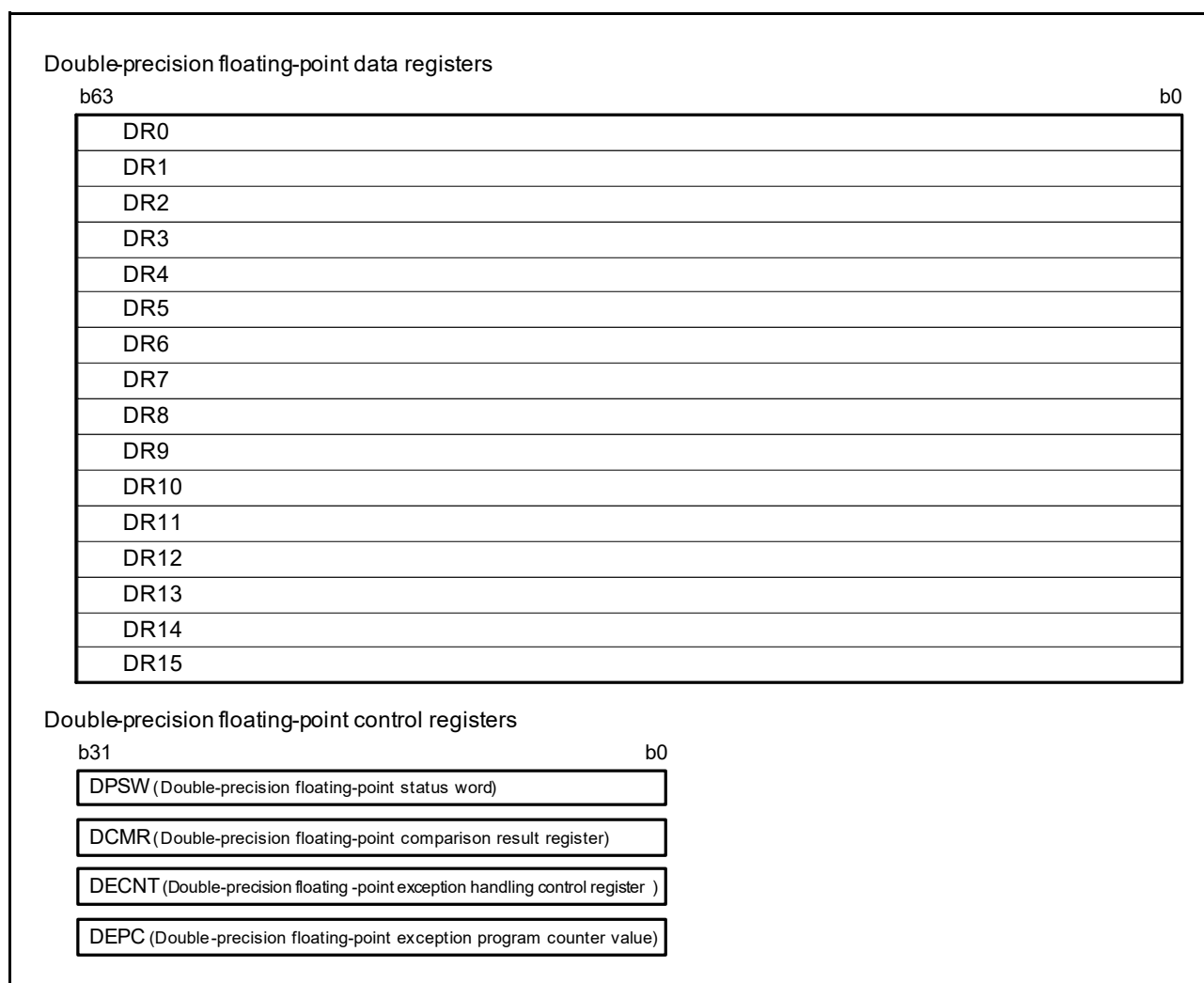


Figure 2.11 Double-Precision Floating-Point Register Set

2.8.2.1 Double-Precision Floating-Point Data Registers (DR0 to DR15)

16 double-precision floating-point data registers with 64-bit width are provided (DR0 to DR15). To specify 32-bit values, use the upper 32 bits (DRH0 to DRH15) or lower 32 bits (DRL0 to DRL15) as separate units.

2.8.2.2 Double-Precision Floating-Point Control Registers

Four double-precision floating-point control registers are provided.

- Double-precision floating-point status word (DPSW)
- Double-precision floating-point comparison result register (DCMR)
- Double-precision floating-point exception handling control register (DECNT)
- Double-precision floating-point exception program counter (DEPC)

(1) Double-Precision Floating-Point Status Word (DPSW)

	b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
	DFS	DFX	DFU	DFZ	DFO	DFV	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

	b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
	—	DEX	DEU	DEZ	DEO	DEV	—	DDN	DCE	DCX	DCU	DCZ	DCO	DCV	DRM[1:0]	—
Value after reset:	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

Bit	Symbol	Bit Name	Description	R/W
b1, b0	DRM[1:0]	Double-Precision Floating-Point Rounding-Mode Setting	b1 b0 0 0: Round to the nearest value 0 1: Round towards 0 1 0: Round towards $+\infty$ 1 1: Round towards $-\infty$	R/W
b2	DCV	Invalid Operation Cause Flag	0: No invalid operation has been encountered. 1: Invalid operation has been encountered.	R/(W) ^{*1}
b3	DCO	Overflow Cause Flag	0: No overflow has occurred. 1: Overflow has occurred.	R/(W) ^{*1}
b4	DCZ	Division-by-Zero Cause Flag	0: No division-by-zero has occurred. 1: Division-by-zero has occurred.	R/(W) ^{*1}
b5	DCU	Underflow Cause Flag	0: No underflow has occurred. 1: Underflow has occurred.	R/(W) ^{*1}
b6	DCX	Inexact Cause Flag	0: No inexact exception has been generated. 1: Inexact exception has been generated.	R/(W) ^{*1}
b7	DCE	Unimplemented Processing Cause Flag	0: No unimplemented processing has been encountered. 1: Unimplemented processing has been encountered.	R/(W) ^{*1}
b8	DDN	0 Flush Bit of Denormalized Number	0: A denormalized number is handled as a denormalized number. 1: A denormalized number is handled as 0. ^{*2}	R/W
b9	—	Reserved	This bit is read as 0. The write value should be 0.	R/W
b10	DEV	Invalid Operation Exception Enable	0: Invalid operation exception is masked. 1: Invalid operation exception is enabled.	R/W
b11	DEO	Overflow Exception Enable	0: Overflow exception is masked. 1: Overflow exception is enabled.	R/W
b12	DEZ	Division-by-Zero Exception Enable	0: Division-by-zero exception is masked. 1: Division-by-zero exception is enabled.	R/W
b13	DEU	Underflow Exception Enable	0: Underflow exception is masked. 1: Underflow exception is enabled.	R/W
b14	DEX	Inexact Exception Enable	0: Inexact exception is masked. 1: Inexact exception is enabled.	R/W
b25 to b15	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b26	DFV	Invalid Operation Flag ^{*3}	0: No invalid operation has been encountered. 1: Invalid operation has been encountered. ^{*8}	R/W
b27	DFO	Overflow Flag ^{*4}	0: No overflow has occurred. 1: Overflow has occurred. ^{*8}	R/W
b28	DFZ	Division-by-Zero Flag ^{*5}	0: No division-by-zero has occurred. 1: Division-by-zero has occurred. ^{*8}	R/W
b29	DFU	Underflow Flag ^{*6}	0: No underflow has occurred. 1: Underflow has occurred. ^{*8}	R/W
b30	DFX	Inexact Flag ^{*7}	0: No inexact exception has been generated. 1: Inexact exception has been generated. ^{*8}	R/W

Bit	Symbol	Bit Name	Description	R/W
b31	DFS	Double-Precision Floating-Point Error Summary Flag	This bit reflects the logical OR of the DFU, DFZ, DFO, and DFV flags.	R

Note 1. When 0 is written to the bit, the setting of the bit will be 0; the bit retains the previous value in response to the writing of 1.

Note 2. Positive denormalized numbers are treated as +0, negative denormalized numbers as -0.

Note 3. When the DEV bit is set to 0, the DFV flag is enabled.

Note 4. When the DEO bit is set to 0, the DFO flag is enabled.

Note 5. When the DEZ bit is set to 0, the DFZ flag is enabled.

Note 6. When the DEU bit is set to 0, the DFU flag is enabled.

Note 7. When the DEX bit is set to 0, the DFX flag is enabled.

Note 8. Once the bit has been set to 1, this value is retained until it is set to 0 by software.

The double-precision floating-point status word (DPSW) indicates the results of double-precision floating-point arithmetic operations.

When the corresponding exception handling enable bits (DE_j) are set to enable processing of the exceptions (DE_j = 1), the DC_j flags can be used by the exception handling routine (the interrupt handling routine with the double-precision floating-point exception as its source) to identify the source of that exception. If handling of an exception is masked (DE_j = 0), the DF_j flag can be used to check for the generation of the exception at the end of a sequence of processing. The DF_j flags operate in an accumulative fashion (j = X, U, Z, O, or V).

The single-precision floating-point status word (FPSW) is neither referred to nor updated in double-precision floating-point arithmetic operations.

DRM[1:0] Bits (Double-Precision Floating-Point Rounding-Mode Setting)

These bits specify the double-precision floating-point rounding-mode.

Explanation of Double-Precision Floating-Point Rounding Modes

- Rounding to the nearest value (the default behavior): An inexact result is rounded to the available value that is closest to the result of a hypothetical calculation with infinite precision. If two available values are equally close, rounding is to the even alternative.
- Rounding towards 0: An inexact result is rounded to the smallest available absolute value; i.e., in the direction of zero (simple truncation).
- Rounding towards +∞: An inexact result is rounded to the nearest available value in the direction of positive infinity.
- Rounding towards -∞: An inexact result is rounded to the nearest available value in the direction of negative infinity.

(1) Rounding to the nearest value is specified as the default mode and returns the most accurate value.

(2) Modes such as rounding towards 0, rounding towards +∞, and rounding towards -∞ are used to ensure precision when interval arithmetic is employed.

DCV Flag (Invalid Operation Cause Flag), DCO Flag (Overflow Cause Flag), DCZ Flag (Division-by-Zero Cause Flag), DCU Flag (Underflow Cause Flag), DCX Flag (Inexact Cause Flag), and DCE Flag (Unimplemented Processing Cause Flag)

Double-precision floating-point exceptions include the five specified in the IEEE754 standard, namely overflow, underflow, inexact, division-by-zero, and invalid operation. For a further double-precision floating-point exception that is generated upon detection of unimplemented processing, the corresponding flag (DCE) is set to 1.

- If an exception or processing that is not implemented is not encountered in the execution of a double-precision floating-point arithmetic instruction other than DABS or DNEG, the corresponding flags become 0.
- When 0 is written to the bit by the MVTDC instruction, the bit is set to 0; the bit retains the previous value when 1 is written by the instruction.

DDN Bit (0 Flush Bit of Denormalized Number)

When this bit is set to 0, a denormalized number is handled as a denormalized number.

When this bit is set to 1, a denormalized number is handled as 0.

DEV Bit (Invalid Operation Exception Enable), DEO Bit (Overflow Exception Enable), DEZ Bit (Division-by-Zero Exception Enable), DEU Bit (Underflow Exception Enable), and DEX Bit (Inexact Exception Enable)

When any of the five floating-point exceptions specified in the IEEE754 standard is generated in the execution of a double-precision floating-point operation instructions, these bits determine whether the CPU will start handling the exception (i.e., whether an interrupt request will be sent to the interrupt controller). When the bit corresponding to an exception is 0, interrupt requests are not generated. When the bit corresponding to an exception is 1, interrupt requests are generated.

DFV Flag (Invalid Operation Flag), DFO Flag (Overflow Flag), DFZ Flag (Division-by-Zero Flag), DFU Flag (Underflow Flag), and DFX Flag (Inexact Flag)

While the exception handling enable bit (DEj) is 0 (exception handling is masked), if any of five floating-point exceptions specified in the IEEE754 standard is generated, the corresponding bit is set to 1.

- When DEj is 1 (exception handling is enabled), the value of the flag remains.
- When the corresponding flag is set to 1, it remains 1 until it is set to 0 by software (Accumulation flag).

DFS Flag (Double-Precision Floating-Point Error Summary Flag)

This bit reflects the logical OR of the DFU, DFZ, DFO, and DFV flags.

The value of this register is not updated when the EHM and EHS bits in the DECNT register are 1.

(2) Double-Precision Floating-Point Comparison Result Register (DCMR)

	b31	b30	b29	b28	b27	b26	b25	b24	b23	b22	b21	b20	b19	b18	b17	b16
	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
	b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
	—	—	—	—	—	—	—	—	—	—	—	—	—	—	—	RES
Value after reset:	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Bit	Symbol	Bit Name	Description	R/W
b0	RES	Double-Precision Floating-Point Compare Instruction Result Flag	0: Condition for comparison was not satisfied. 1: Condition for comparison was satisfied.	R/W
b31 to b1	—	Reserved	These bits are read as 0. The write value should be 0.	R/W

(3) Double-Precision Floating-Point Exception Handling Control Register (DECNT)

[illegible]

Bit	Symbol	Bit Name	Description	R/W
b0	EHM	Double-Precision Floating-Point Exception Information Preservation Mode	0: Mode in which information on the sources of exceptions is not preserved. Information on the generation of double-precision floating-point exceptions is not preserved when they occur. 1: Mode in which information on the generation of exceptions is preserved. When a double-precision floating-point exception is generated and an interrupt request is sent to the interrupt controller, the EHS bit is changed to 1 to preserve the information that an exception has been generated.	R/W
b15 to b1	—	Reserved	These bits are read as 0. The write value should be 0.	R/W
b16	EHS	Double-Precision Floating-Point Exception Information Preservation Status	0: Exception information is not being preserved. 1: Exception information is being preserved. Updating of the following registers stops when the EHM and EHS bits are 1, and interrupts due to double-precision floating-point exceptions do not occur. • Double-precision floating-point status word • Double-precision floating-point exception program counter Writing 0 to this bit releases the exception information at the time the exception was generated from preservation.	R/W
b31 to b17	—	Reserved	These bits are read as 0. The write value should be 0.	R/W

(4) Double-Precision Floating-Point Exception Program Counter (DEPC)

The DEPC register holds the value of the program counter for the instruction that caused the most recent exception when a double-precision floating-point exception is generated and an interrupt request is sent to the interrupt controller. This register is read-only. The value of this register is not updated when the value of the EHS bit in the DECNT register is 1 while that of the EHM bit in the same register is also 1.

2.8.3 Data Types (for the Double-Precision Floating-Point Coprocessor)

The double-precision floating-point coprocessor can handle double-precision floating-point numbers.

In addition, it can handle single-precision floating-point numbers with the DTOF and FTOD instructions, and 32-bit integers with the DROUND, DTOI, DTOU, ITOD and UTOI instructions.

For single-precision floating-point numbers, refer to section 2.4.2, Single-Precision Floating-Point Numbers, and for 32-bit integers, refer to section 2.4.1, Integer.

2.8.3.1 Double-Precision Floating-Point Numbers

The double-precision floating-point number is compliant with that specified in the IEEE754 standard. Operands of this type can be used in fifteen double-precision floating-point arithmetic instructions: DABS, DADD, DCMPCm, DDIV, DMUL, DNEG, DROUND, DSUB, DSQRT, DTOF, DTOI, DTOU, FTOD, ITOD, and UTOI.

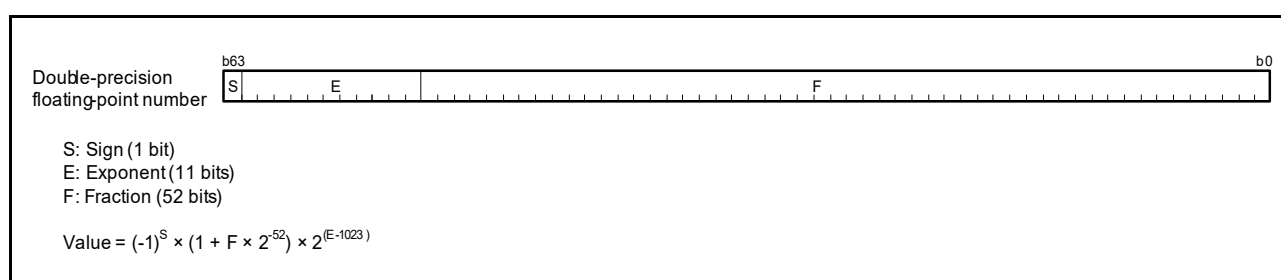


Figure 2.12 Double-Precision Floating-Point Number

The double-precision floating-point number can represent the values listed below.

$0 < E < 2047$ (normal numbers)

$E = 0$ and $F = 0$ (signed zero)

$E = 0$ and $F > 0$ (denormalized numbers)*1

$E = 2047$ and $F = 0$ (infinity (∞))

$E = 2047$ and $F > 0$ (Not-a-Number (NaN))

Note 1. The number is treated as 0 when the DPSW.DDN bit is 1. When the DDN bit is 0, an unimplemented processing exception is generated.

2.8.4 Data Arrangement (for the Double-Precision Floating-Point Coprocessor)

The double-precision floating-point coprocessor can handle 32-bit and 64-bit values.

For 32-bit values, refer to section 2.5.4, Data Arrangement, and section 2.5.1, Switching the Endian.

2.8.4.1 Arrangement of Data in the Double-Precision Floating-Point Registers

Figure 2.13 shows the relationship between the sizes of registers DR0 to DR15 and the bit numbers.

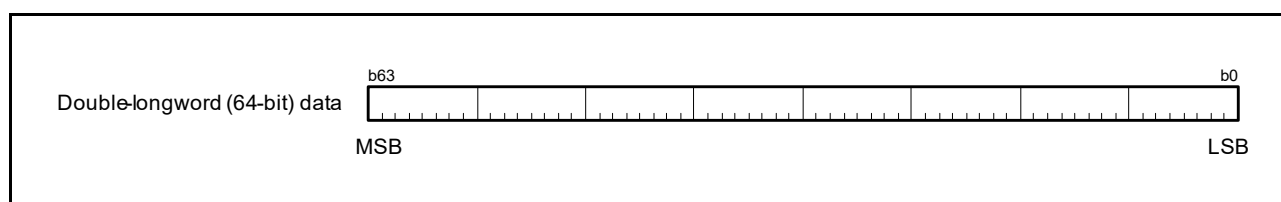


Figure 2.13 Data Arrangement in Registers

2.8.4.2 Arrangement of Data for Double-Precision Floating-Point Numbers in Memory

A double-precision floating-point number is always represented as a double-longword (64 bits) in memory. The data arrangement is selectable as little endian or big endian. Figure 2.14 shows the arrangement of data in memory.

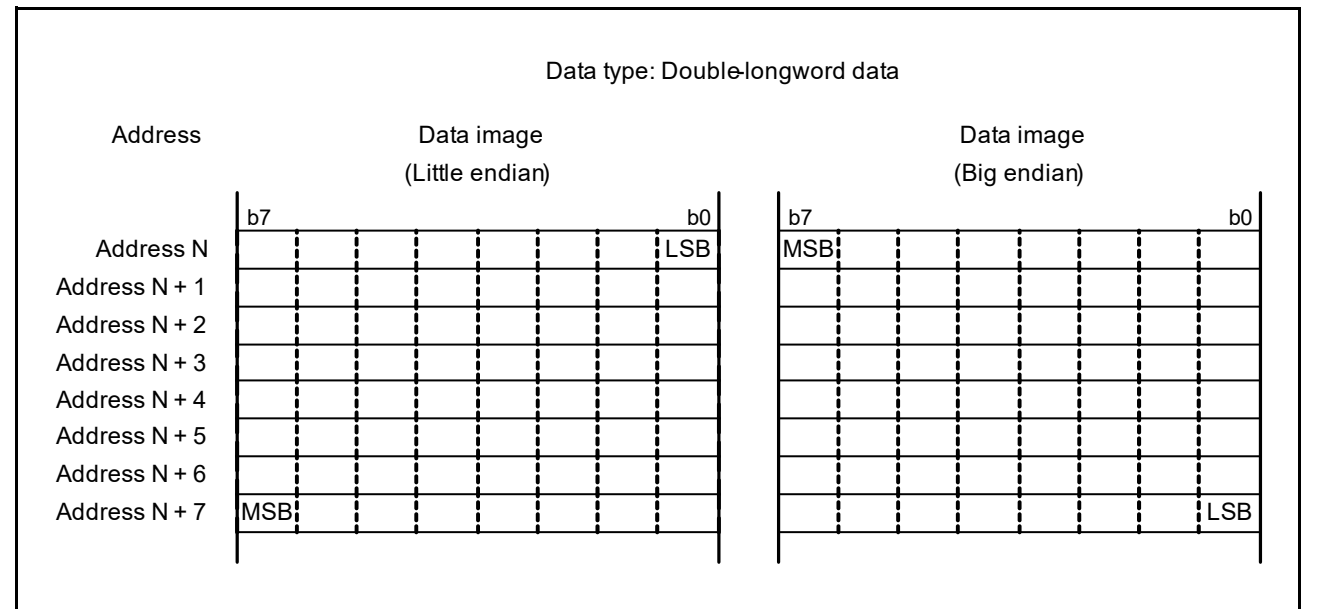


Figure 2.14 Data Arrangement in Memory

2.8.5 Switching the Endian (for the Double-Precision Floating-Point Coprocessor)

The operation in access with 64-bit values by double-precision floating-point processing instructions depends on the endian setting. Table 2.13 to Table 2.16 show operations in access for each endian and in the cases of reading and writing.

Address exceptions are generated in response to 64-bit access to addresses that are not on 32-bit boundaries.

In the tables,

- LLL indicates bits D7 to D0 of the double-precision floating-point data registers,
- LLH indicates bits D15 to D8 of the double-precision floating-point data registers,
- LHL indicates bits D23 to D16 of the double-precision floating-point data registers,
- LHH indicates bits D31 to D24 of the double-precision floating-point data registers,
- HLL indicates bits D39 to D32 of the double-precision floating-point data registers,
- HLH indicates bits D47 to D40 of the double-precision floating-point data registers,
- HHL indicates bits D55 to D48 of the double-precision floating-point data registers, and
- HHH indicates bits D63 to D56 of the double-precision floating-point data registers.

Double-precision floating-point data register: DRm	D63 to D56		D55 to D48		D47 to D40		D39 to D32	
	HHH		HHL		HLH		HLL	
	D31 to D24		D23 to D16		D15 to D8		D7 to D0	
	LHH		LHL		LLH		LLL	

Table 2.13 64-Bit Read Operations when Little Endian has been Selected

Operation Address of src	Reading a 64-bit unit from address 0	Reading a 64-bit unit from address 4	Reading a 64-bit unit from address 8
Address 0	Transfer to LLL	—	—
Address 1	Transfer to LLH	—	—
Address 2	Transfer to LHL	—	—
Address 3	Transfer to LHH	—	—
Address 4	Transfer to HLL	Transfer to LLL	—
Address 5	Transfer to HLH	Transfer to LLH	—
Address 6	Transfer to HHL	Transfer to LHL	—
Address 7	Transfer to HHH	Transfer to LHH	—
Address 8	—	Transfer to HLL	Transfer to LLL
Address 9	—	Transfer to HLH	Transfer to LLH
Address A	—	Transfer to HHL	Transfer to LHL
Address B	—	Transfer to HHH	Transfer to LHH
Address C	—	—	Transfer to HLL
Address D	—	—	Transfer to HLH
Address E	—	—	Transfer to HHL
Address F	—	—	Transfer to HHH

Table 2.14 64-Bit Read Operations when Big Endian has been Selected

Operation Address of src	Reading a 64-bit unit from address 0	Reading a 64-bit unit from address 4	Reading a 64-bit unit from address 8
Address 0	Transfer to HHH	—	—
Address 1	Transfer to HHL	—	—
Address 2	Transfer to HLH	—	—
Address 3	Transfer to HLL	—	—
Address 4	Transfer to LHH	Transfer to HHH	—
Address 5	Transfer to LHL	Transfer to HHL	—
Address 6	Transfer to LLH	Transfer to HLH	—
Address 7	Transfer to LLL	Transfer to HLL	—
Address 8	—	Transfer to LHH	Transfer to HHH
Address 9	—	Transfer to LHL	Transfer to HHL
Address A	—	Transfer to LLH	Transfer to HLH
Address B	—	Transfer to LLL	Transfer to HLL
Address C	—	—	Transfer to LHH
Address D	—	—	Transfer to LHL
Address E	—	—	Transfer to LLH
Address F	—	—	Transfer to LLL

Table 2.15 64-Bit Write Operations when Little Endian has been Selected

Operation Address of dest	Writing a 64-bit unit to address 0	Writing a 64-bit unit to address 4	Writing a 64-bit unit to address 8
Address 0	Transfer to LLL	—	—
Address 1	Transfer to LLH	—	—
Address 2	Transfer to LHL	—	—
Address 3	Transfer to LHH	—	—
Address 4	Transfer to HLL	Transfer to LLL	—
Address 5	Transfer to HLH	Transfer to LLH	—
Address 6	Transfer to HHL	Transfer to LHL	—
Address 7	Transfer to HHH	Transfer to LHH	—
Address 8	—	Transfer to HLL	Transfer to LLL
Address 9	—	Transfer to HLH	Transfer to LLH
Address A	—	Transfer to HHL	Transfer to LHL
Address B	—	Transfer to HHH	Transfer to LHH
Address C	—	—	Transfer to HLL
Address D	—	—	Transfer to HLH
Address E	—	—	Transfer to HHL
Address F	—	—	Transfer to HHH

Table 2.16 64-Bit Write Operations when Big Endian has been Selected

Operation Address of dest	Writing a 64-bit unit to address 0	Writing a 64-bit unit to address 4	Writing a 64-bit unit to address 8
Address 0	Transfer to HHH	—	—
Address 1	Transfer to HHL	—	—
Address 2	Transfer to HLH	—	—
Address 3	Transfer to HLL	—	—
Address 4	Transfer to LHH	Transfer to HHH	—
Address 5	Transfer to LHL	Transfer to HHL	—
Address 6	Transfer to LLH	Transfer to HLH	—
Address 7	Transfer to LLL	Transfer to HLL	—
Address 8	—	Transfer to LHH	Transfer to HHH
Address 9	—	Transfer to LHL	Transfer to HHL
Address A	—	Transfer to LLH	Transfer to HLH
Address B	—	Transfer to LLL	Transfer to HLL
Address C	—	—	Transfer to LHH
Address D	—	—	Transfer to LHL
Address E	—	—	Transfer to LLH
Address F	—	—	Transfer to LLL

2.9 Operation of Instructions

2.9.1 Restrictions on RMPA and String-Manipulation Instructions

2.9.1.1 Transfer Size and Data Prefetching

The RMPA instruction and the string-manipulation instructions (SCMPU, SMOVB, SMOVF, SMOVU, SSTR, SUNTIL, and SWHILE instructions) transfer data in longword units to speed up the reading of data from and writing of data to the memory. If the last of the data to be processed is less than a longword, data transfer proceeds with the sizes described below:

- RMPA, SSTR, SUNTIL, and SWHILE instructions: Size specified by the size specifier
- SCMPU, SMOVB, SMOVF, and SMOVU instructions: Byte

Additionally, in the above processing, the RMPA instruction and the string-manipulation instructions other than the SSTR instruction (that is, the SCMPU, SMOVB, SMOVF, SMOVU, SUNTIL, and SWHILE instructions) prefetch data when reading data from the memory. Data is prefetched from the prefetching start position with three bytes as the upper limit. The prefetching start positions of each operation are shown below.

- RMPA instruction: The multiplicand address specified by R1, and the multiplier address specified by R2
- SCMPU instruction: The source address specified by R1 for comparison, and the destination address specified by R2 for comparison
- SUNTIL and SWHILE instructions: The destination address specified by R1 for comparison
- SMOVB, SMOVF, and SMOVU instructions: The source address specified by R2 for transfer

2.9.1.2 Access to the External Space

Although the external space has a per-area ending-switching facility (only for data), the allocation of data to be handled by RMPA or string-manipulation instructions (SCMPU, SMOVB, SMOVF, SMOVU, SSTR, SUNTIL, and SWHILE instructions) to an area where the endian differs from that of the chip is prohibited, and operation is not guaranteed if this restriction is not observed. If data to be handled by RMPA or string-manipulation instructions are allocated to the external space, they must be allocated to areas where the endian setting is the same as that for the chip.

2.9.1.3 Access to I/O Registers

The allocation of data to be handled by RMPA or string-manipulation instructions (SCMPU, SMOVB, SMOVF, SMOVU, SSTR, SUNTIL, and SWHILE instructions) to I/O registers is prohibited, and operation is not guaranteed if this restriction is not observed.

2.10 Numbers of Cycles

2.10.1 Instruction and Numbers of Cycles

Table 2.17 to Table 2.25 show the numbers of cycles in operation of each instruction. The listed numbers of cycles for access to memory are the numbers of cycles during no-wait access. The operands in the table below indicate the following meanings.

#IMM: Immediate

flag: bit, flag

Rs, Rs2, Rd, Rd2, Ri, Rb: General-purpose register

As, Ad: Accumulator

CR: Control register

dsp: displacement

pcdsp: displacement

Table 2.17 Numbers of Cycles for Arithmetic/logic Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Arithmetic/logic instructions (register-register, immediate- register)	• {ABS, NEG, NOT} "Rd"/"Rs, Rd" • {ADC, MAX, MIN, ROTL, ROTR} "#IMM, Rd"/"Rs, Rd" • ADD "#IMM, Rd"/"Rs, Rd"/"#IMM, Rs, Rd"/"Rs, Rs2, Rd" • {AND, MUL, OR, SUB, XOR} "#IMM, Rd"/"Rs, Rd"/"Rs, Rs2, Rd" • {CMP, TST} "#IMM, Rs"/"Rs, Rs2" • NOP • {ROL, ROR, SAT} "Rd" • SBB "Rs, Rd" • {SHAR, SHLL, SHLR} "#IMM, Rd"/"Rs, Rd"/"#IMM, Rs, Rd"	1
	• DIV "#IMM, Rd"/"Rs, Rd"	3 to 20*1
	• DIVU "#IMM, Rd"/"Rs, Rd"	2 to 18*1
	• {EMUL, EMULU} "#IMM, Rd"/"Rs, Rd"	2
	• SATR	3
	Arithmetic/logic instructions (memory source operand)	3
	• {ADC, ADD, AND, MAX, MIN, MUL, OR, SBB, SUB, XOR} "[Rs], Rd"/"dsp[Rs], Rd" • {CMP, TST} "[Rs], Rs2"/"dsp[Rs], Rs2"	
	• DIV "[Rs], Rd / dsp[Rs], Rd"	5 to 22*1
	• DIVU "[Rs], Rd / dsp[Rs], Rd"	4 to 20*1
	• {EMUL, EMULU} "[Rs], Rd"/"dsp[Rs], Rd"	4
	• RMPA.B	6+7×floor(n/4)+4×(n%4) n: Number of processing bytes*2
	• RMPA.W	6+5×floor(n/2)+4×(n%2) n: Number of processing words*2
	• RMPA.L	6+4n n: Number of processing longwords

Note 1. The numbers of cycles for the dividing instruction varies according to the divisor and dividend.

Note 2. floor (x): Max. integer that is smaller than x.

Table 2.18 Numbers of Cycles for Transfer Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Transfer instructions (register-register, immediate-register)	• MOV "#IMM, Rd"/"Rs, Rd" • {MOVU, REVL, REWV} "Rs, Rd" • SCCnd "Rd" • {STNZ, STZ} "#IMM, Rd"/"Rs, Rd"	1
	• XCHG "Rs, Rd"	2
Transfer instructions (load operation)	• {MOV, MOVU} "[Rs], Rd"/"dsp[Rs], Rd"/"[Rs+], Rd"/"[-Rs], Rd"/"[Ri, Rb], Rd" • MOVL "Rs, Rd" • POP "Rd"	Throughput: 1 Latency: 2* ¹
	• POPC "CR"	Throughput: 3 Latency: 4* ¹
	• POPM "Rd-Rd2"	Throughput: n Latency: n+1 n: Number of registers* ¹ , * ²
Transfer instructions (store operation)	• MOV "Rs, [Rd]"/"Rs, dsp[Rd]"/"Rs, [Rd+]/"Rs, [-Rd]"/"Rs, [Ri, Rb]"/"IMM, dsp[Rd]"/"IMM, [Rd]" • PUSH "Rs" • PUSHC "CR" • SCCnd "[Rd]"/"dsp[Rd]" • MOVCO "Rs, [Rd]"	1
	• PUSHM "Rs-Rs2"	n n: Number of registers* ³
Transfer instructions (memory-register)	• XCHG "[Rs], Rd"/"dsp[Rs], Rd"	2
Transfer instructions (memory-memory)	• MOV "[Rs], [Rd]"/"dsp[Rs], [Rd]"/"[Rs], dsp[Rd]"/"dsp[Rs], dsp[Rd]" • PUSH "[Rs]"/"dsp[Rs]"	3
Transfer instructions (bit field)	• {BFMOV, BFMOVZ} "#IMM, #IMM, #IMM, R, R"	1

Note 1. When the load data is used by the subsequent instruction, the numbers of cycles described as "latency" is counted as the numbers of cycles for the memory load instruction. For the cycles other than the memory load instruction, the numbers of cycles described as "throughput" is counted.

Note 2. The POPM instruction is converted into multiple load operations. The processing is the same as the one for the load operations of the MOV instruction, where the operation is repeated for the number of specified registers.

Note 3. The PUSHM instruction is converted into multiple store operations. The processing is the same as the one for the store operations of the MOV instruction, where the operation is repeated for the number of specified registers.

Table 2.19 Numbers of Cycles for Bit Manipulation Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Bit manipulation instructions (register)	• {BCLR, BNOT, BSET} "#IMM, Rd"/"Rs, Rd" • BMCnd "#IMM, Rd" • BTST "#IMM, Rs"/"Rs, Rs2"	1
Bit manipulation instructions (memory source operand)	• {BCLR, BNOT, BSET} "#IMM, [Rd]"/"IMM, dsp[Rd]"/"Rs, [Rd]"/"Rs, dsp[Rd]" • BMCnd "#IMM, [Rd]"/"IMM, dsp[Rd]" • BTST "#IMM, [Rs]"/"IMM, dsp[Rs]"/"Rs, [Rs2]"/"Rs, dsp[Rs2]"	3

Table 2.20 Numbers of Cycles for Branch Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Branch instructions	• BCnd "pcdsp" • {BRA, BSR} "pcdsp"/"Rs" • {JMP, JSR} "Rs"	Branch taken: 3 Branch not taken: 1
	• RTE	6
	• RTFI	3
	• RTS	5
	• RTSD "#IMM"	5
	• RTSD "#IMM, Rd-Rd2"	Throughput: $n < 5?5:1+n$ Latency: $n < 4?5:2+n$ n: Number of registers*1

?: Conditional operator

Note 1. When the load data is used by the subsequent instruction, the numbers of cycles described as "latency" is counted as the numbers of cycles for the memory load instruction. For the cycles other than the memory load instruction, the numbers of cycles described as "throughput" is counted.

Table 2.21 Numbers of Cycles for Single-Precision Floating-Point Operation Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Single-precision floating-point operation instructions (register-register, immediate-register)	• {FADD, FSUB} "#IMM, Rd"/"Rs, Rd"/"Rs, Rs2, Rd"	2
	• FCMP "#IMM, Rs"/"Rs, Rs2"	1
	• FDIV "#IMM, Rd"/"Rs, Rd"	16
	• FMUL "#IMM, Rd"/"Rs, Rd"/"Rs, Rs2, Rd"	2
	• FSQRT "Rs, Rd"	16
	• {FTOI, ROUND, ITOF} "Rs, Rd"	2
	• {FTOU, UTOF} "Rs, Rd"	2
Single-precision floating-point operation instructions (memory source operand)	• {FADD, FSUB} "[Rs], Rd"/"dsp[Rs], Rd"	4
	• FCMP "[Rs], Rs2"/"dsp[Rs], Rs2"	3
	• FDIV "[Rs], Rd"/"dsp[Rs], Rd"	18
	• FMUL "[Rs], Rd"/"dsp[Rs], Rd"	4
	• FSQRT "[Rs], Rd"/"dsp[Rs], Rd"	18
	• {FTOI, ROUND, ITOF} "[Rs], Rd"/"dsp[Rs], Rd"	4
	• {FTOU, UTOF} "[Rs], Rd"/"dsp[Rs], Rd"	4

Table 2.22 Numbers of Cycles for DSP Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
DSP instructions	• {EMULA, EMACA, EMSBA, MULLH, MULHI, MULLO, MACLH, MACHI, MACLO, MSBLH, MSBHI, MSBLO} "Rs, Rs2, Ad" • {MVFACHI, MVFACMI, MVFACLO, MVFACGU} "#IMM, As, Rd" • {MVTACHI, MVTACLO, MVTACGU} "Rs, Ad" • {RDACW, RDACL, RACW, RACL} "#IMM, Ad"	1

Table 2.23 Numbers of Cycles for String Manipulation Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
String manipulation instructions*1	• SCMPU	$2+4 \times \text{floor}(n/4)+4 \times (n\%4)$ n: Number of comparison bytes*2
	• SMOVB	$n > 3 ? 6+3 \times \text{floor}(n/4)+3 \times (n\%4): 2+3n$ n: Number of transfer bytes*2
	• SMOVF, SMOVU	$2+3 \times \text{floor}(n/4)+3 \times (n\%4)$ n: Number of transfer bytes*2
	• SSTR.B	$2+\text{floor}(n/4)+n\%4$ n: Number of transfer bytes*2
	• SSTR.W	$2+\text{floor}(n/2)+n\%2$ n: Number of transfer words*2
	• SSTR.L	$2+n$ n: Number of transfer longwords
	• SUNTIL.B, SWHILE.B	$3+3 \times \text{floor}(n/4)+3 \times (n\%4)$ n: Number of comparison bytes*2
	• SUNTIL.W, SWHILE.W	$3+3 \times \text{floor}(n/2)+3 \times (n\%2)$ n: Number of comparison words*2
	• SUNTIL.L, SWHILE.L	$3+3 \times n$ n: Number of comparison longwords

?: Conditional operator

Note 1. Each of the SCMPU, SMOVU, SWHILE, and SUNTIL instructions ends the execution regardless of the specified cycles, if the end condition is satisfied during execution.

Note 2. floor (x): Max. integer that is smaller than x.

Table 2.24 Numbers of Cycles for System Manipulation Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
System manipulation instructions	• {CLRPSW, SETPSW} "flag"	1
	• MVTC "#IMM, CR"/"Rs, CR"	
	• MVFC "CR, Rd"	
	• MVTIPL "#IMM"	
	• RTE	6
	• RTFI	3

Table 2.25 Numbers of Cycles for Instructions for Register Bank Save Function

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Instructions for register bank save function	• SAVE "#IMM"/"R"	1
	• RSTR "#IMM"/"R"	3 to 6

2.10.2 Instruction and Numbers of Cycles (for the Double-Precision Floating-Point Coprocessor)

Table 2.26 shows the numbers of cycles for the double-precision floating-point related instructions. The listed numbers of cycles for access to memory are the numbers of cycles during no-wait access. The operands in the table below indicate the following meanings.

#IMM: Immediate

Rs, Rd: General-purpose register

dsp: displacement

DRs, DRs2, DRd, DRd2: Double-precision floating-point data register

DRHs, DRHd: Upper 32 bits in the double-precision floating-point data register

DRLs, DRLd: Lower 32 bits in the double-precision floating-point data register

DCRs, DCRs2, DCRd, DCRd2: Double-precision floating-point control register

Table 2.26 Numbers of Cycles for the Double-Precision Floating-Point Related Instructions

Instruction	Mnemonic (indicates the common operation when the size is omitted)	Numbers of Cycles
Double-precision floating-point processing instructions	• {DADD, DSUB} "DRs, DRs2, DRd"	4*1
	• {DABS, DNEG} "DRs, DRd"	1
	• DCMPPcm "DRs, DRs2"	4*1
	• DDIV "DRs, DRs2, DRd"	33*1
	• DMUL "DRs, DRs2, DRd"	5*1
	• DSQRT "DRs, DRd"	33*1
	• {DTOF, DTOI, DTOU, DROUND} "DRs, DRd"	4*1
	• {FTOD, ITOD, UTOD} "Rs, DRd"	4*1
Double-precision floating-point data transfer instructions	• DMOV "Rs, DRHd" / "Rs, DRLd" / "DRHs, Rd" / "DRLs, Rd" / "DRs, DRd" / "#IMM, DRHd" / "#IMM, DRLd"	1
	• DMOV "DRs, [Rd]" / "DRs, dsp[Rd]"	1
	• DMOV "[Rs], DRd" / "dsp[Rs], DRd"	Throughput: 1 Latency: 2*2
	• DPUSHM "DRs-DRs2" / "DCRs-DCRs2"	n n: Number of registers*4
	• DPOPM "DRd-DRd2" / "DCRd-DCRd2"	Throughput: n Latency: n + 1 n: Number of registers*2, *3
	• MVFDC "DCRs, Rd"	1
	• MVFDR	1
	• MVTDC "Rs, DCRd"	1

Note 1. The numbers of cycles for reference to the results of arithmetic operations by double-precision floating-point data transfer instructions and the DABS and DNEG instructions are the listed numbers of cycles plus 1.

Note 2. When the load data is used by the subsequent instruction, the numbers of cycles described as "latency" is counted as the numbers of cycles for the memory load instruction. For the cycles other than the memory load instruction, the numbers of cycles described as "throughput" is counted.

Note 3. The DPOPM instruction is converted into multiple load operations. The processing is the same as the one for the load operations of the DMOV instruction, where the operation is repeated for the number of specified registers.

Note 4. The DPUSHM instruction is converted into multiple store operations. The processing is the same as the one for the store operations of the DMOV instruction, where the operation is repeated for the number of specified registers.

2.10.3 Numbers of Cycles for Response to Interrupts

Table 2.27 lists numbers of cycles taken by processing for response to interrupts.

Table 2.27 Numbers of Cycles for Response to Interrupts

Type of Interrupt Request/Details of Processing	Fast Interrupt	Other Interrupts
ICU Judgment of priority order	2 cycles	
CPU Numbers of cycles from notification to acceptance of the interrupt request	N cycles (varies with the instruction being executed at the time the interrupt was received)	
CPU Pre-processing by hardware Saving the current PC and PSW values in RAM (or in control registers in the case of the fast interrupt) Reading of the vector Branching to the start of the exception handling routine	4 cycles	6 cycles

Times calculated from the values in Table 2.27 will be applicable when access to memory from the CPU is processed with no waiting. This MCU has a RAM that allows no-wait access and code flash memory incorporating a ROM Cache. Numbers of cycles for response to interrupts can be minimized by placing program code (and vectors) in code flash memory and the stack in RAM. Furthermore, place the addresses where the exception handling routine start on 8-byte boundaries.

For information on the numbers of cycles from notification to acceptance of the interrupt request, indicated by N in the table above, see Table 2.17 to Table 2.26.

The timing of interrupt acceptance depends on the execution state of the instruction. For more information on this, see section 14.3.1, Acceptance Timing and Saved PC Value.

2.11 Usage Note

2.11.1 Notes on Self-Diagnosis of the RAM for the Save Register Banks

The save register banks in this MCU are configured of RAM. As the save register banks are equipped with a buffer, data may be read from the buffer rather than from the memory cells of the RAM when the same address is to be read by a RSTR instruction after a write operation by the SAVE instruction. When running self-diagnosis of the RAM in the save register banks, confirm that the data have actually been written to the memory by following the procedure below so that data will not be read from the buffer.

- (1) Write data to the bank targeted for diagnosis with the SAVE instruction.
- (2) Write data to a bank different from the bank in the procedure (1) with the SAVE instruction.
- (3) Read the data from the bank in the procedure (1) with the RSTR instruction.